PX4Lockstep.jl: Deterministic PX4 Lockstep SITL with a Julia-Owned Hybrid Simulation Engine

Bharat Tak
Wingtra AG
bharat.tak@wingtra.com

February 2, 2026

Abstract

`PX4Lockstep.jl` is a lockstep software-in-the-loop (SITL) harness for PX4 in which the autopilot is advanced through a minimal C ABI on an injected integer-microsecond clock, while a Julia runtime owns the hybrid simulation. The runtime builds a multi-rate union event axis from subsystem cadences, executes all discrete components in a fixed boundary protocol, and integrates the continuous plant between boundaries under sample-and-hold inputs.

The resulting execution model is explicit enough to be reviewed and tested: for a fixed aircraft specification, initial plant state, and stochastic seeds, repeated runs under a fixed execution environment (single-threaded execution and pinned dependencies) produce the same boundary times and the same boundary-aligned traces (bytewise in the best case). Closed-loop missions can be recorded and replayed as open-loop plant experiments, isolating plant/integrator changes from controller behavior.

The simulator provides modular models for rigid-body multirotor dynamics, actuation/propulsion, battery and power-network coupling, atmosphere, and sampled wind/turbulence, plus scenario orchestration and structured logging. This paper documents the implementation code-faithfully, states the determinism threat model, and summarizes current verification coverage and planned evaluation.

Revision history

- **2026-02-02** — Added commander-lite in the lockstep C runtime and routed high-level commands via `set_cmd!`. Moved status/control/armed topics out of Julia injection, made uORB bridge reads allocation-free, and reorganized the test suite into domain subfolders with stricter mission-finished integration checks.

Table of Contents

1 Scope, terminology, and design goals

This paper documents `PX4Lockstep.jl`, a PX4 lockstep SITL driver and hybrid drone simulator written in Julia. The implementation has two coupled layers:

1. a thin Julia wrapper around `libpx4_lockstep` that steps PX4 on an injected integer-microsecond clock and publishes/subscribes uORB messages, and
2. a deterministic Julia simulation stack that owns the hybrid timeline, plant dynamics, environment, scenarios, and logging.

1.1 Conceptual model

Figure 1 shows the conceptual organization. The runtime engine advances an integer-microsecond event axis. At each event boundary it executes discrete-time *sources* (scenario, wind, estimator, PX4 lockstep) to update a held `SimBus`. The continuous plant integrates between boundaries under sample-and-hold inputs constructed from the bus. Algebraic *derived outputs* (e.g., battery telemetry) are evaluated at boundaries and published into the bus for PX4 and for logging.

1.2 Terminology and conceptual layering

The codebase is modular, but the paper uses a small number of terms consistently. Table 1 defines the ones needed to reason about determinism.

1.3 Design goals and reader takeaways

The primary goal is not maximum physics fidelity; it is *reviewable determinism* for controller-in-the-loop experiments:

- **Deterministic scheduling:** all boundary times are integer microseconds and the engine advances on an explicit union event axis.
- **Hybrid semantics:** discrete sources update only at event boundaries; the plant integrates with sample-and-hold inputs between boundaries.
- **Minimal trust boundary:** PX4 runs as a black box behind the lockstep ABI; all "truth" models live in Julia.
- **Reproducible analysis:** closed-loop runs can be recorded and replayed deterministically to compare plant-side changes.

Even if a reader never opens the repository, the intended takeaways are the four invariants below, which jointly define the execution model:

1. **Integer time contract:** all subsystem cadences and delays are exact multiples of 1 µs (otherwise the code errors).
2. **Fixed boundary protocol:** all discrete updates at t_k occur in a single canonical order (section 2.3), producing one well-defined bus value `bus(t_k)`.
3. **Sample-and-hold integration:** the plant integrates over $[t_k, t_{k+1})$ using a single held input packet derived from `bus(t_k)`.

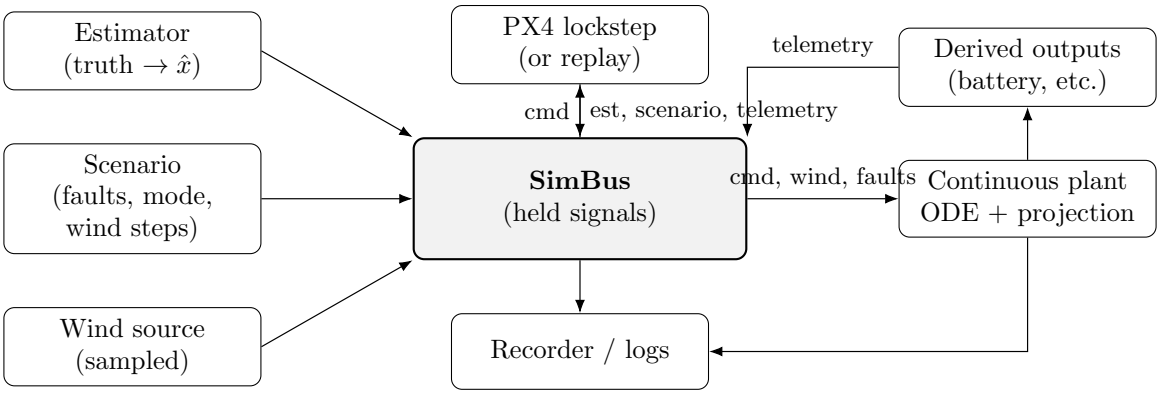

Figure 1: Conceptual structure. The engine schedules discrete sources on an integer-microsecond event axis and integrates the continuous plant between boundaries under sample-and-hold inputs.

Term	Meaning in this paper (as implemented)
time axis	A sorted vector of integer microseconds owned by one subsystem (e.g., PX4 tick times, wind tick times). Built by `Runtime.Timeline` (`src/sim/Runtime/Timeline.jl`).
event axis	The sorted unique union of all time axes. The engine integrates only on this axis.
boundary	A specific event time t_k where discrete sources may update the bus and where logging may occur.
source	A discrete-time component with `Runtime.update!(src,bus,plant,t_us)` (scenario, wind source, estimator, PX4 wrapper, replay samplers).
bus	A mutable packet `Runtime.SimBus` holding piecewise-constant signals (cmd, wind, faults, estimates, telemetry) between boundaries.
plant	The continuous-time state integrated by the engine: rigid body plus (optionally) actuator, rotor, and power states (section 6).
derived outputs	Algebraic outputs computed from the current plant state and held inputs at boundaries via `plant_outputs(...)` (e.g. battery telemetry).

Table 1: Terminology and conceptual layering used throughout the paper.

4. **Record/replay falsifiability:** boundary-aligned traces are recorded and replayed with exact axis alignment, enabling deterministic open-loop plant comparisons (section 14).

Whenever a statement below depends on a specific implementation choice, we point to the corresponding Julia entrypoint (module/function) rather than relying on prose in repository documentation. All file paths are written relative to the `px4_lockstep_julia` repository root.

Category	Symbol	Meaning	Units
Time	t_k	k-th boundary time on the union event axis (real-valued seconds).	s
	$t_{k,\mu s}$	Same boundary time stored as integer microseconds.	μs
	Δt_k	Step length $t_{k+1} - t_k$.	s
	$\mathcal{T}_{ap}, \mathcal{T}_{wind}, \mathcal{T}_{log}, \mathcal{T}_{evt}$	Autopilot / wind / log / union event axes (sorted integer-μs vectors).	μs
Plant	$x(t), x_k$	Continuous plant state at time t and at boundary t_k.	mixed
Input	u_k	Held plant input packet derived from bus(t_k); held on $[t_k, t_{k+1})$.	mixed
Frames	$\mathbf{p}_{ned}, \mathbf{v}_{ned}$	Position and velocity in NED.	m, m/s
	ω_{body}	Angular rate in body/FRD.	rad/s
	$\mathbf{q}_{bn}, \mathbf{R}_{bn}$	Attitude quaternion and DCM for Body $\to$ NED.	–
Contact	$\phi(\mathbf{p})$	Gap function; $\phi > 0$ above ground plane.	m
	δ	Penetration depth $\max(0, -\phi)$.	m
	F_n	Normal contact force magnitude (upward).	N
	$\mathbf{v}_t,\ v_t$	Tangential (ground-plane) velocity and its magnitude.	m/s
	$\mathbf{F}_t$	Tangential friction force.	N
Power	s	State-of-charge (fraction).	–
	v_1	Thevenin polarization voltage state.	V
	T	Battery temperature.	°C
	r_0, r_1, C_1	Thevenin parameters (series resistance, polarization resistance/-capacitance).	$\Omega, \Omega, \mathrm{F}$
	V_{bus}, I_{bus}	Bus voltage and current.	V, A

Table 2: Notation used throughout the paper. Scalars are italic, vectors/matrices are bold, and frame tags are explicit in subscripts.

2 Deterministic runtime engine

The canonical run loop is `Sim.Runtime.Engine` (`src/sim/Runtime/Engine.jl`). It advances an integer-microsecond timeline, calls discrete-time *sources* at event boundaries, and integrates the continuous-time plant between boundaries.

2.1 Timebase and hybrid event timeline

All scheduling uses integer microseconds (`UInt64`). The helper `dt_to_us(dt_s)` (`src/sim/Runtime/Timeline.jl`) converts a `Float64` interval in seconds to microseconds with an exactness requirement:

$$\left| dt\_s - 10^{-6}\,\mathrm{round}(10^6 dt\_s) \right| \le 10^{-15}. \tag{1}$$

If the conversion is not exact (i.e. the requested interval is not an integer multiple of 1 µs), the code throws.

Timeline axes. The runtime constructs several named time axes, each a sorted vector of microseconds (`src/sim/Runtime/Timeline.jl`): autopilot ticks, wind ticks, logging ticks, scenario *AtTime* boundaries, and an optional "physics" axis. The global event axis is the sorted unique union of all axis times, including both endpoints. The engine always integrates on this union axis: any subsystem boundary becomes an integration boundary.

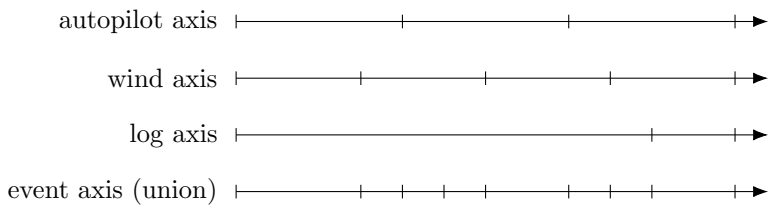

Figure 2: Multi-rate scheduling: each subsystem owns a time axis, and the engine integrates on their union (the event axis).

2.2 TOML configuration: timeline axes

The timeline parameters described above are configured in the `[timeline]` block of the aircraft spec (`TimelineSpec` in `src/sim/Aircraft/Spec.jl`, parsed by `src/sim/Aircraft/TOMLIO.jl`):

```
[timeline]
t_end_s = 20.0
dt_autopilot_s = 0.004
dt_wind_s = 0.001
dt_log_s = 0.01
dt_phys_s = 0.001 # optional; omit (or set to null) to disable extra physics boundaries
```

Parameter semantics (what the code does).

- `t_end_s`: inclusive simulation horizon. The engine always starts at $t_0 = 0$ and forces both t_0 and t_{end} into the union event axis.
- `dt_autopilot_s`: cadence of the autopilot axis (PX4 stepping, estimator publication, plant discontinuities).
- `dt_wind_s`: cadence of the wind axis. Wind is updated only on these ticks and held between them.
- `dt_log_s`: cadence of logging ticks. The implementation additionally appends a final log sample at t_{end} even when dt_{log} does not evenly divide the horizon (`build_timeline`).
- `dt_phys_s`: optional additional periodic boundaries. When omitted, the "physics" axis degenerates to $\{t_0, t_{end}\}$ and does not add boundaries beyond the other axes.

Mapping to the engine axes. Each interval is converted to integer microseconds by `dt_to_us` (section 2.1), giving

$$\Delta t_{ap} \leftrightarrow \texttt{dt\_autopilot\_s}, \quad \Delta t_{wind} \leftrightarrow \texttt{dt\_wind\_s}, \quad \Delta t_{log} \leftrightarrow \texttt{dt\_log\_s}, \quad \Delta t_{phys} \leftrightarrow \texttt{dt\_phys\_s}. \tag{2}$$

Scenario *AtTime* boundaries are *not* configured in `[timeline]`; they are discovered from the scenario source via the `event_times_us` protocol hook and injected into the union axis by `build_timeline_for_run`.

2.3 Boundary protocol and sample-and-hold semantics

At each event time t_k the engine executes boundary stages in a fixed canonical order (`src/sim/Runtime/BoundaryProtocol.jl`, applied by `process_events_at!` in `src/sim/Runtime/Engine.jl`):

1. **Scenario**: update high-level commands (arm/mission/RTL), landed flag, faults, and optional wind disturbances.
2. **Wind**: if the wind axis is due, step the wind source and publish the held `bus.wind_ned` sample.
3. **Derived outputs**: if the plant implements `plant_outputs`, evaluate algebraic outputs (notably battery telemetry) and publish them into the bus.
4. **Estimator**: if the autopilot axis is due, publish `bus.est` (truth injection + noise/bias/delay).
5. **Telemetry**: optional deterministic inspection hooks (autopilot-axis only).
6. **Autopilot**: if the autopilot axis is due, step PX4 (or replay) and publish `bus.cmd`.
7. **Plant discontinuities**: if the autopilot axis is due, apply boundary-time plant updates (notably snapping *direct* actuators via `plant_on_autopilot_tick`).
8. **Logging**: if the log axis is due, record a pre-step snapshot and emit structured logs.

Why this order? The boundary protocol is part of the simulator semantics, not an implementation detail. At each boundary t_k the engine applies a discrete transition

$$(\mathrm{bus}_{k^-}, x_k) \xrightarrow{\text{boundary stages}} (\mathrm{bus}_{k^+}, x_k^+),$$

where x_k is the continuous plant state at the boundary and x_k^+ includes any boundary-time discontinuities (e.g. actuator snaps). The held plant input over the next interval is then defined by

$$u_k \coloneqq \mathrm{PlantInput}(\mathrm{bus}_{k^+}), \qquad u(t) = u_k \text{ for } t \in [t_k, t_{k+1}).$$

Reordering stages changes which signals are held over which intervals and therefore changes the meaning of the simulation.

The chosen ordering enforces the following invariants at PX4 ticks:

- **No stale estimator inputs:** when the autopilot axis is due, PX4 consumes an estimator packet constructed from truth at the same boundary time t_k.
- **No stale telemetry:** algebraic telemetry (e.g. battery status) is computed from (t_k, x_k, u_k) and published before stepping PX4, so uORB injection reflects the current plant state.
- **Command-to-physics causality:** PX4 is stepped exactly once at t_k, producing the unique command u_k that is held over $[t_k, t_{k+1})$.
- **Snap-after-command:** boundary-time discontinuities are applied after `bus.cmd` is published, so snapped actuator state is a deterministic function of the newly produced command.
- **Logs correspond to integration inputs:** logging occurs last so a snapshot at t_k corresponds to the same bus packet that seeds integration over $[t_k, t_{k+1})$.

Changing this ordering is therefore a semantic change (not "just refactoring"): for example, stepping PX4 before the estimator would make PX4 consume a stale estimate; logging before PX4 would decouple recorded commands from the plant inputs that followed.

Scenario cadence. The scenario stage is currently executed at *every* boundary on the union event axis, not only on the scenario axis. This makes hybrid `When(...)`-style logic responsive at the earliest possible boundary while preserving deterministic ordering (see the note in `src/sim/Runtime/BoundaryProtocol.jl` and the implementation in `process_events_at!`).

Wind disturbance consistency. If the scenario updates an additive wind disturbance at a boundary where the wind axis is *not* due, the engine applies the disturbance delta immediately to the held wind sample so the piecewise-constant wind input remains consistent across axes.

Sample-and-hold. Between event times, all bus values are treated as sample-and-hold. The plant is integrated over $[t_k, t_{k+1})$ with a single input packet

$$u_k \coloneqq \{\text{actuator command},\ \text{wind},\ \text{fault state}\}, \tag{3}$$

constructed from the bus at t_k (`PlantInput` in `src/sim/Plant.jl`).

2.4 Integration stepping and post-step projection

Let $x(t)$ be the continuous plant state and $f(t, x, u)$ the plant RHS functor. At each interval the engine calls

$$x_{k+1} \leftarrow \text{Integrate}(f,\ t_k,\ x_k,\ u_k,\ \Delta t_k) \tag{4}$$

where $\Delta t_k = (t_{k+1} - t_k)\, 10^{-6}$ seconds, and the integrator is selected by the aircraft spec (`src/sim/Integrators.jl`, `src/sim/Aircraft/Build.jl`).

Hybrid interval integration (contact-aware plants). Most plants can be advanced by calling `Integrators.step_integrator` once per union-axis interval. Ground contact, however, is inherently hybrid: touchdown and liftoff introduce discontinuities that are not representable as a smooth ODE without either (i) stiff penalty forces or (ii) explicit event handling. The runtime therefore supports an optional protocol hook

```
plant_integrate_interval(model,integrator,t0_us,x0,u,dt_us)
```

(`src/sim/PlantInterface.jl`). When implemented, the engine delegates the *entire* interval advance to this method.

Free flight vs. contact-constrained stepping (current ground-contact implementation). For the coupled multirotor plant, `plant_integrate_interval` is implemented when `PlantSpec.contact` is `FlatGroundConstraintContact`. The interval integrator acts as a deterministic regime switcher:

- **Airborne segments (smooth dynamics):** the plant is advanced under the *free* dynamics f_{free} (the same model but with `NoContact()`) using the configured integrator (RK4/RK23/RK45, etc.). Near the ground, the next airborne chunk can be capped to O(time-to-impact) so that a descending crossing of the guard is bracketed reliably.
- **Impact events (discontinuous map):** if a lookahead airborne chunk brackets a descending crossing of $z = 0$, the TOI is localized to integer microseconds by deterministic bisection and an impact map is applied at that internal time (position projection + velocity reset with restitution and optional tangential friction). Multiple impacts can be handled inside one outer interval, but the code caps the count to a fixed maximum to avoid pathological bounce storms.

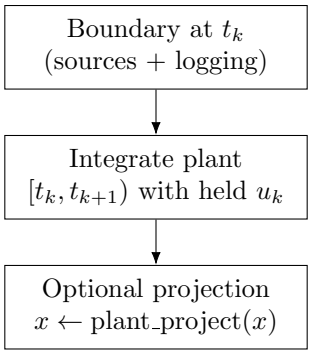

Figure 3: Engine rhythm: boundary processing on the union event axis, then continuous plant integration over the next interval under sample-and-hold inputs.

- **Grounded segments (constraint/impulse stepping):** while the state is a ground candidate, the plant advances in fixed substeps of `h_contact_us`. Each substep uses a single evaluation of f_{free} (explicit Euler) followed by a closed-form, 1-contact impulse solve (normal + Coulomb friction) and a position-only stabilization on z. In this grounded regime the configured integrator is *not* used; the time-stepping loop is deliberately fixed-step and deterministic.

The full contact math and the exact update equations are documented in section 10.4.2.

Interval metadata and logging. Because impacts can occur at internal TOI times $\tau \in (t_k, t_{k+1})$, the interval integrator may optionally return per-interval metadata (impact count, maximum Δv, and its timestamp). The engine accumulates this metadata into latched bus signals for logging (`bus.impact_dv_ned`, `bus.impact_time_us`, `bus.impact_count`) and also computes a simple "equivalent acceleration" estimate at the autopilot tick rate (`bus.impact_accel_est_ned`); see `Runtime.Engine._accumulate_interval_impacts!`.

Some state variables have hard physical bounds (e.g. rotor speed $\omega \geq 0$, battery SOC in $[0, 1]$). The engine therefore supports an optional post-step projection hook `plant_project(model,x)` that clamps or projects state deterministically after each accepted interval. The coupled multirotor plant implements this projection (`src/sim/PlantModels/CoupledMultirotor.jl`).

Record → replay (high level). The same engine can run in a mode that records boundary-aligned bus/plant streams, and a replay mode that replaces "live" sources with trace samplers. This converts a closed-loop PX4 run into a deterministic open-loop plant replay for integrator/model comparisons.

3 Determinism contract, threat model, and evaluation plan

The runtime in section 2 makes the execution model explicit. This section (i) defines what "determinism" means for this project, (ii) states a threat model for when the claim is defensible, and (iii) summarizes what is already verified in the repository and what remains to be evaluated.

3.1 What "determinism" means here

We treat determinism as a *trace-level* property of the simulator under a fixed execution environment. Let $\mathcal{S}$ denote the full experiment specification: the aircraft configuration (TOML), the initial plant

state, and the seeds for any stochastic sources, together with the exact software stack that executes the floating-point path (Julia version, `Manifest.toml`, and relevant runtime environment variables).

A run maps $\mathcal{S}$ to a set of ordered, boundary-aligned traces: commands on the autopilot axis, wind on the wind axis, optional scenario/fault outputs on the event axis, and plant/telemetry snapshots on the log axis (see the Tier-0 recording bundle in section 14.3).

Definition (trace determinism under a fixed environment). Fix an execution environment E (single-threaded execution, pinned dependencies, stable CPU/libm). The simulator is deterministic with respect to E if two executions with identical $\mathcal{S}$ produce:

1. the same event axis $\{t_k\}$ (integer microseconds, same ordering), and
2. identical trace payloads at each recorded timestamp (bytewise identity when serialized is the strongest form; in practice we also report max-norm error metrics for convenience).

When E changes (different OS/CPU/libm or dependency drift), we do *not* claim bytewise identity a priori; instead we treat this as a *reproducibility* experiment and report drift envelopes explicitly.

Decomposed properties (what the design enforces vs. what is empirical). With the above scope, the codebase targets three nested properties:

1. **Scheduling determinism:** the same set of event times $\{t_k\}$ is generated from the same configuration, and the engine visits them in a deterministic order (section 2.1).
2. **Discrete-transition determinism:** at each boundary time t_k, all sources update the bus in a single canonical order, yielding one unambiguous bus(t_k) (section 2.3).
3. **Numerical determinism:** given identical held inputs and a fixed floating-point execution environment, plant integration produces identical $x(t_k)$ at every boundary time.

The first two are architectural (by construction once time is integer microseconds and ordering is fixed). The third is conditional on floating-point and dependency controls and is therefore verified empirically by record/replay trace comparisons and regression tests.

3.2 Mechanisms used in the implementation

The implementation contains several explicit mechanisms intended to make determinism reviewable:

- **Integer-microsecond timebase:** all axes are stored as `UInt64` microseconds and the timeline builder rejects non-exact cadences (`dt_to_us`; section 2.1).
- **Union event axis:** the engine integrates only on the sorted unique union of subsystem axes, so a discrete update can never occur "inside" an integration step (figure 2).
- **Fixed boundary protocol:** a single canonical stage ordering defines what the autopilot consumes and what input is held over the next interval (section 2.3).
- **Sample-and-hold bus:** all plant inputs are derived from a held `SimBus` packet at boundaries (section 2.3).
- **Dedicated RNG ownership for stochastic sources:** live wind and live estimator sources carry their own RNG instances and document the rule "no RNG usage outside of `update!`" (`src/sim/Sources/Wind.jl`, `src/sim/Sources/Estimator.jl`).

- **Deterministic iteration counts in key algebraic solvers:** the coupled plant bus-voltage solve uses staged deterministic strategies with fixed iteration counts (section 9.5).
- **Axis-aligned record/replay:** recorded streams are validated to match their intended axes exactly (`_require_axis_match!`), and replay sources sample by deterministic binary search over the recorded axis vectors (section 14).
- **Microsecond-quantized adaptive substeps:** adaptive Runge–Kutta integrators optionally quantize accepted substeps to integer microseconds (`quantize_us=true` by default in `src/sim/Integrators.jl`) to reduce drift from floating time accumulation.

3.3 TOML configuration: deterministic seeding

All stochastic sources are expected to be seeded explicitly through the aircraft spec. The seed is configured under `[aircraft]`:

```
[aircraft]
name = "iris" # optional human-readable identifier
seed = 1 # integer seed for deterministic RNG ownership
```

Parameter semantics (what the code does).

- `name`: used only for identification/logging; it does not affect dynamics.
- `seed`: used to seed deterministic RNGs during build. In the default builder (`src/sim/Aircraft/Build.jl`), the wind source uses `seed` and the noisy estimator uses `seed+1`. If `seed` is not specified, `AircraftSpec()` defaults to `1`.

This is a narrow but explicit contract: deterministic randomness requires the simulator to own the RNG state and to treat seeds as part of the experiment configuration.

3.4 Threat model for determinism

Even with a deterministic execution model, determinism can be undermined by factors outside the scheduling layer. The paper therefore makes an explicit threat model:

In-scope (what the design intends to control).

- Discrete-event ordering and held-input semantics (fully controlled by the engine).
- RNG-based sources, provided they use dedicated RNG state and fixed seeds (controlled by the source constructors and aircraft spec).
- Data-dependent loop termination, where the code uses fixed-iteration schemes for critical algebraic steps (e.g. bus solve) or where floating-point decisions are deterministic given a fixed environment.

Out-of-scope / risks (must be controlled by the run harness or explicitly disclaimed).

- **Threading and nondeterministic reductions.** The simulator is intended to run single-threaded, but the code does not enforce this globally. Any multi-threaded dependency (e.g. BLAS, linear algebra kernels, reductions) can introduce nondeterministic floating-point association.

- **Cross-platform floating-point differences.** Bitwise identity across CPU architectures, OS-/libm versions, or compiler flags is not guaranteed in IEEE 754 arithmetic. The project currently targets repeatability within a controlled environment; cross-platform repeatability should be treated as an empirical question.
- **Dependency drift.** If package versions change (Julia version, `Manifest.toml` content), numerical behavior can change even if the code is unchanged.
- **Iteration order and hash stability.** Determinism can be broken by unordered iteration (e.g. `Dict` traversal) or randomized hashing across Julia versions. The implementation avoids unordered iteration in core loops; any new code must be careful about iteration order.
- **Math flags and fused operations.** Compiler flags or macros such as `@fastmath` can enable reassociation or fused-multiply-add, which may change results across platforms or builds. These are avoided in the core simulation path.
- **Sorting stability assumptions.** Any use of sorting as a deterministic tie-breaker must guarantee a stable order and explicit keying; otherwise equal keys can lead to nondeterministic outcomes across versions.
- **PX4 black-box behavior.** PX4 is treated as an external component. While lockstep stepping removes wall-clock timing drift, internal PX4 behavior can still depend on floating-point implementation details and configuration.

Required controls for determinism claims. To make determinism claims academically defensible, runs intended for determinism evaluation *must* satisfy the following:

- set `JULIA_NUM_THREADS=1` and disable BLAS threading (e.g. `OPENBLAS_NUM_THREADS=1`, `MKL_NUM_THREADS=1`, `VECLIB_MAXIMUM_THREADS=1`),
- execute with the repository's `Project.toml` and `Manifest.toml` pinned (no dependency drift),
- use explicit seeds for any live stochastic sources (wind, noisy estimator), and
- report the Julia version, OS, CPU, and relevant environment variables alongside results.

3.5 Evidence already present in the repository

Even without an "evaluation" section with plots, the repository already contains regression tests that operationalize determinism at the trace level:

- `test/verification/record_replay_engine.jl` constructs minimal plants and asserts: (i) replay integration matches an analytic solution for a piecewise-constant input, (ii) record mode captures axis-aligned Tier-0 traces, and (iii) record $\rightarrow$ reload $\rightarrow$ replay reproduces the logged plant state at every log tick to tight tolerances (position/velocity/attitude/body-rate/rotor/battery terms on the order of 10^{-12} or smaller). This is the operative determinism contract in the current test suite.
- `test/verification/compare_integrators.jl` runs deterministic integrator sweeps on a fixed recording, validating solver ordering and error metric wiring.
- Multiple unit tests validate model components that directly influence determinism (e.g. battery surface interpolation and scaling, motor mapping, and gyro/rotor sign conventions).

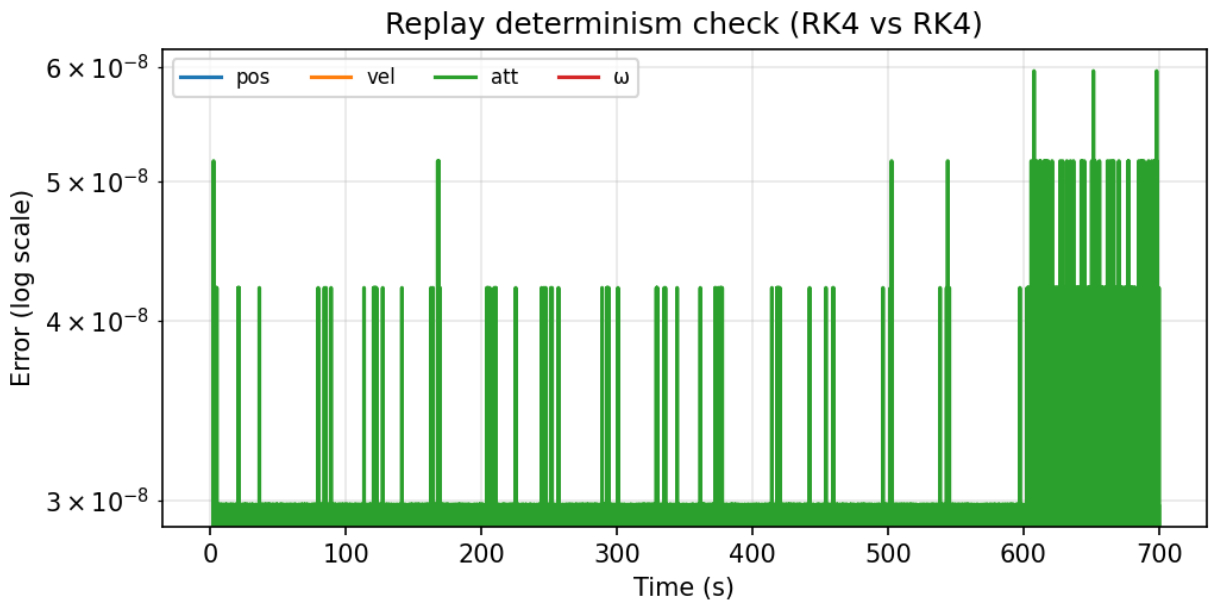

Figure 4: Determinism check: two replay runs of the same recording using the same integrator (RK4) produce negligible drift over 700 s.

These tests are limited (short horizon, controlled scenarios), but they provide a concrete starting point: determinism is checked by comparing boundary-aligned traces, not by subjective visual similarity.

3.6 Evaluation plan (results to be added)

A "proper" paper version should extend the above test-style checks into a small evaluation section. The following experiments can be added without changing the architecture:

1. **Replay fidelity (closed-loop → open-loop):** record a short closed-loop mission, then replay the plant using the recorded `:cmd`, `:wind_base_ned`, and scenario traces, and report that Tier-0 log streams match within tolerance.

2. **Repeated-run determinism:** run the same recording replay multiple times and verify identical Tier-0 outputs within tolerance (optionally also report bytewise hashes when strict controls are used).

3. **Cross-machine repeatability (if feasible):** replay the same Tier-0 file on a second machine/OS and report drift envelopes, explicitly noting whether results are bitwise-identical or only numerically close.

4. **Cost trade-offs:** measure wall-clock overhead as a function of event-axis density (union-axis refinement), log cadence, and adaptive-integrator tolerance.

5. **Sensitivity to numerical choices:** use the existing integrator sweep tooling (section 14) to report how solver choice changes plant trajectories under identical recorded inputs, and quantify the effect of `quantize_us` on adaptive error control.

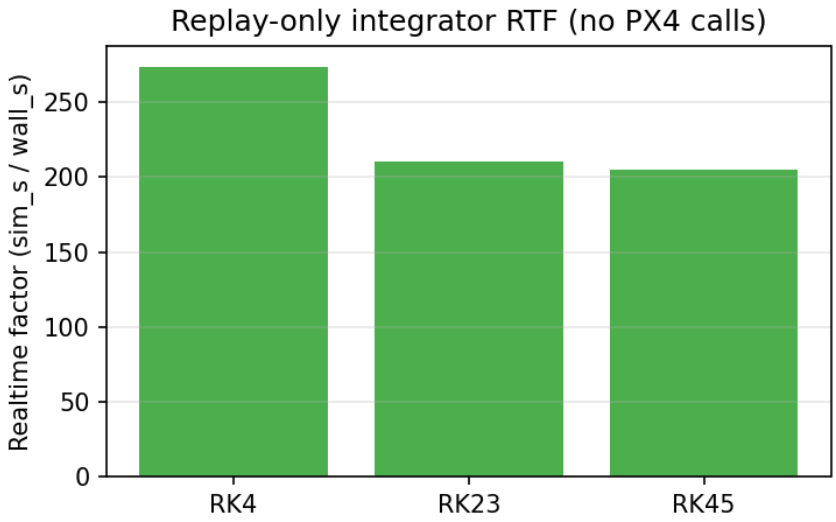

Figure 5: Replay-only realtime factor for a 700 s Iris mission recording (no PX4 calls during the replay sweep). Run configuration is summarized in section C.

Where the current implementation makes arbitrary choices (e.g. fixed iteration counts, simple contact model, simplified aero), those choices should be stated as such and evaluated for their impact on determinism, stability, and fidelity rather than defended as universally correct.

4 Evidence summary (current repository)

The full validation results are presented later (section 15), but a short summary here anchors the key claims early:

- **Determinism replay drift:** two replay runs of the same recording produce negligible drift over 700 s (figure 4).
- **Replay performance:** replay-only realtime factor for a 700 s mission is summarized in figure 5 (no PX4 calls during replay).
- **Integrator verification:** analytic/invariant checks for SHO, pendulum, Kepler, and torque-free rigid-body cases are tabulated in table 5.
- **Record/replay sweep:** end-to-end Iris record/replay integrator sweep errors are reported in table 6.

These results are intentionally lightweight but they are executable and traceable to scripts in the repository, making them suitable as regression anchors.

5 Frames and sign conventions

The simulator follows PX4's conventional world frame:

- **World/inertial frame**: NED (North–East–Down). Position and velocity are stored as $\mathbf{p}_{ned}$ and $\mathbf{v}_{ned}$ in meters and m/s.
- **Body frame**: FRD (Forward–Right–Down). Angular rate is stored as $\omega_{body} = (p, q, r)$ in rad/s.

5.1 Quaternion convention

Attitude is stored as a unit quaternion $\mathbf{q}_{bn}$ representing the rotation *Body* $\rightarrow$ *NED* in (w, x, y, z) order (`src/sim/Types.jl`). The corresponding direction cosine matrix is denoted $\mathbf{R}_{bn}$ (`quat_to_dcm`).

The rigid-body kinematic equation implemented in `src/sim/RigidBody.jl` is

$$\dot{\mathbf{q}}_{bn} = \tfrac{1}{2}\,\mathbf{q}_{bn} \otimes \begin{bmatrix} 0 \\ \omega_{body} \end{bmatrix}, \tag{5}$$

where $\otimes$ is the Hamilton product (`quat_mul`). Quaternions are explicitly normalized during state updates to limit numerical drift.

5.2 Home origin and NED-to-LLA injection

Several parts of the simulator (atmosphere sampling and PX4 uORB state injection) require converting local NED coordinates into latitude/longitude/altitude (LLA). The implementation uses a *spherical Earth* small-angle approximation with a fixed radius R_E = `EARTH_RADIUS_M` $= 6.378137 \times 10^6$ m (`src/sim/Autopilots.jl`):

$$\Delta\varphi = \frac{p_{ned,x}}{R_E}, \qquad \Delta\lambda = \frac{p_{ned,y}}{R_E \cos(\text{deg2rad}(\varphi_0))}, \tag{6}$$

$$\varphi = \varphi_0 + \text{rad2deg}(\Delta\varphi), \qquad \lambda = \lambda_0 + \text{rad2deg}(\Delta\lambda), \tag{7}$$

$$h_{msl} = h_0 - p_{ned,z}, \tag{8}$$

where $(\varphi_0, \lambda_0, h_0)$ is the configured home origin and $p_{ned,z}$ is positive down.

Validity range. This flat-Earth approximation is intended for local missions on the order of $\sim$10 km from the home origin. Longitude uses a fixed $\cos(\varphi_0)$ scale factor and altitude is linear; large-area navigation requires a geodesic model. See section 18 for the rationale and extension plan.

Radius consistency note. For LLA injection we use the WGS84 equatorial radius ($R_E = 6.378137 \times 10^6$ m). The default spherical-gravity model now uses the same value (via `Autopilots.EARTH_RADIUS_M`) for `gravity_r0_m` in the environment spec (section 10). Both are configurable if a different convention is required.

Angle units. Throughout the codebase, fields named `*_deg` are stored in degrees. When a trigonometric function is applied to a `*_deg` value, the implementation uses degree-aware functions (e.g. `cosd`) or an explicit `deg2rad` conversion. This is done at the call site, not globally; readers should assume trig is in radians unless a `*d` helper or an explicit `deg2rad` appears.

TOML configuration. The home origin is configured via the `[home]` block (`Types.WorldOrigin`, aliased as `Autopilots.HomeLocation`):

```
[home]
lat_deg = 47.397742
lon_deg = 8.545594
alt_msl_m = 488.0
```

Parameter mapping.

- `lat_deg` $\to \varphi_0$ in degrees.
- `lon_deg` $\to \lambda_0$ in degrees.
- `alt_msl_m` $\to h_0$ in meters above mean sea level.

These parameters affect only coordinate conversions and altitude-dependent environment queries; they do not change the local NED rigid-body dynamics directly.

5.3 Propulsor axis convention

Each propulsor i has a unit axis vector $\mathbf{a}_{b,i}$ expressed in body/FRD axes. See `Vehicles.PropulsorLayout` and `Vehicles.QuadrotorParams`. The thrust force applied to the rigid body is defined as

$$\mathbf{F}_{b,i} = -T_i\,\mathbf{a}_{b,i}, \tag{9}$$

so the "typical" multirotor case (thrust along $-$body-z) uses $\mathbf{a}_{b,i} = (0, 0, 1)$. Rotor axes from specs are normalized during aircraft build (`src/sim/Aircraft/Build.jl`).

The propulsion layer produces a signed reaction torque magnitude per rotor. The rigid-body model applies it as a moment about the propulsor axis:

$$\tau_{b,i}^{\text{reaction}} = Q_i\,\mathbf{a}_{b,i}. \tag{10}$$

Rotor direction bookkeeping. The multirotor configuration distinguishes:

- `rotor_dir[i]`: the *reaction torque sign on the body* used to sign Q_i.
- Rotor spin direction: opposite to the reaction torque sign (used for rotor gyroscopic coupling, section 7).

6 Continuous-time plant state and integrators

The engine integrates a *full plant state* rather than only rigid-body states. This makes variable-step integration meaningful: actuator dynamics, rotor-speed dynamics, and battery dynamics are integrated alongside the rigid body.

6.1 Plant state and held input packet

The rigid-body state (`RigidBodyState` in `src/sim/RigidBody.jl`) is

$$x_{rb} \coloneqq \{\mathbf{p}_{ned},\ \mathbf{v}_{ned},\ \mathbf{q}_{bn},\ \omega_{body}\}. \tag{11}$$

The full continuous plant state (`PlantState` in `src/sim/Plant.jl`) extends this with actuator and powertrain states:

$$x \coloneqq \{x_{rb},\ \mathbf{y}_m,\ \dot{\mathbf{y}}_m,\ \mathbf{y}_s,\ \dot{\mathbf{y}}_s,\ \mathbf{\Omega},\ \text{power}\}, \tag{12}$$

$$\mathbf{y}_m \in \mathbb{R}^{12},\ \mathbf{y}_s \in \mathbb{R}^8,\ \mathbf{\Omega} \in \mathbb{R}^N. \tag{13}$$

Motor/servo outputs are stored in fixed-size arrays to match the PX4 lockstep ABI (12 motor channels, 8 servo channels), while N is the number of *physical* propulsors.

The plant consumes a piecewise-constant input packet (`PlantInput`) held over each engine interval:

$$u \coloneqq \{\texttt{cmd},\ \mathbf{w}_{ned},\ \texttt{faults}\}, \tag{14}$$

where `cmd` is the latest actuator command, $\mathbf{w}_{ned}$ is the latest sampled wind velocity, and `faults` is a compact fault mask (e.g. motor disable bitmask, battery connected flag).

6.2 Fixed-step and adaptive integrators

The integrators in `src/sim/Integrators.jl` are dependency-light and operate directly on `RigidBodyState` or `PlantState`:

- **Euler**: forward Euler (debug).
- **RK4**: classic fixed-step 4th-order Runge–Kutta.
- **RK23**: Bogacki–Shampine 3(2) with adaptive step.
- **RK45**: Dormand–Prince 5(4) with adaptive step.

Adaptive methods compute an error estimate from the difference between the "high" and "low" order solutions. The default error norm is an ℓ_∞-style scaled maximum over rigid-body groups (position, velocity, body rates) plus a geodesic attitude error on $SO(3)$:

$$e_q = \frac{2\arccos(|\langle q_{hi}, q_{lo}\rangle|)}{\max(\texttt{atol\_att\_rad}, \epsilon)}. \tag{15}$$

By default, non-rigid-body plant states (actuator, rotor speed, battery) do *not* influence adaptivity. Setting `plant_error_control=true` enables additional error terms provided the corresponding absolute tolerances are finite.

Microsecond quantization of adaptive substeps. Both adaptive integrators support `quantize_us=true` (default), which quantizes internal substep sizes to integer microseconds to preserve the engine's integer-time contract. This is a determinism trade-off: quantization slightly perturbs the step-size controller and can degrade error-control optimality near stiff-ish dynamics (e.g. contact-adjacent transients). Its impact should be reported in evaluations.

Engine-level note. The engine resets integrator internal step-size history at the start of every event interval (`reset!(integrator)` inside `Engine.step_to_next_event!`). This maximizes determinism isolation between intervals but may reduce adaptive efficiency compared to carrying history across intervals.

6.3 TOML configuration: integrator selection and tolerances

The integrator described in section 6.2 is selected in the `[plant]` block (`PlantSpec.integrator`, parsed by `_parse_integrator` in `src/sim/Aircraft/TOMLIO.jl`). The parser accepts either a string shorthand:

```
[plant]
integrator = "RK4" # "Euler" | "RK4" | "RK23" | "RK45"
```

or a table form for adaptive solvers with explicit tolerances:

```
[plant]
integrator = { kind="RK45",
  rtol_pos=1e-7, atol_pos=1e-6,
  rtol_vel=1e-7, atol_vel=1e-6,
  rtol_omega=1e-7, atol_omega=1e-6,
  atol_att_rad=1e-6,
  plant_error_control=false,
  h_min=1e-6,
  max_substeps=50000,
  quantize_us=true
}
```

Parameter mapping to the adaptive error model. The tolerance keys map directly to the error computation used by RK23/RK45:

- `rtol_pos`, `atol_pos`: relative/absolute tolerances for the position group $\mathbf{p}_{ned}$.
- `rtol_vel`, `atol_vel`: relative/absolute tolerances for the velocity group $\mathbf{v}_{ned}$.
- `rtol_omega`, `atol_omega`: tolerances for body rates ω_{body}. (The TOML parser also accepts Unicode omega keys via TOML escapes, e.g. `rtol_\u03C9` and `atol_\u03C9`.)
- `atol_att_rad`: absolute tolerance for the attitude geodesic error e_q in section 6.2.
- `plant_error_control`: when `true`, include non-rigid-body plant states (actuators, rotor speed, SOC, v_1, temperature) in the adaptive error norm if their absolute tolerances are finite.

Step-size controls.

- `h_min`: lower bound on internal adaptive substeps (seconds).
- `max_substeps`: hard cap on internal substeps per engine interval; exceeding it throws.
- `quantize_us`: when `true`, accepted adaptive substeps are quantized to integer microseconds to reduce drift against the engine's integer-time axis (section 2.1).

Any omitted fields take constructor defaults from the selected integrator's constructor (`src/sim/Integrators.jl`).

7 Rigid-body multirotor dynamics

The baseline vehicle model is `Sim.Vehicles.GenericMultirotor`, a rigid-body multirotor with configurable propulsor geometry. It is intentionally low-fidelity but allocation-free and stable for closed-loop testing.

7.1 Translational dynamics

The position derivative is simply

$$\dot{\mathbf{p}}_n = \mathbf{v}_n. \tag{16}$$

The total thrust force in body coordinates is formed by summing per-rotor thrust vectors. Each propulsor has a body-frame axis $\mathbf{a}_{b,i}$ (`QuadrotorParams.rotor_axis_body`) with the convention

$$\mathbf{F}_{b,i} = -T_i\,\mathbf{a}_{b,i}, \tag{17}$$

so for a classic multirotor with thrust along $-z_b$, $\mathbf{a}_{b,i} = (0, 0, 1)$. The total body force is

$$\mathbf{F}_b = \sum_{i=1}^{N} \mathbf{F}_{b,i}, \qquad \mathbf{F}_n = \mathbf{R}_{bn}\, \mathbf{F}_b. \tag{18}$$

A simple linear drag term is applied in NED based on air-relative velocity

$$\mathbf{v}_{rel} = \mathbf{v}_n - \mathbf{w}_n, \qquad \mathbf{F}_{drag} = -c_d\, \mathbf{v}_{rel}, \tag{19}$$

where c_d is `QuadrotorParams.linear_drag` and $\mathbf{w}_n$ is the held wind sample from the runtime bus.

Finally, gravity is supplied by the environment model as an acceleration vector $\mathbf{g}_n$ (positive down in NED). The velocity derivative is

$$\dot{\mathbf{v}}_n = \frac{1}{m} \left(\mathbf{F}_n + \mathbf{F}_{drag}\right) + \mathbf{g}_n. \tag{20}$$

7.2 Rotational dynamics with rotor gyroscopics

Moments from propulsors have two contributions: (i) thrust at a lever arm, and (ii) reaction torque about the propulsor axis. With body-frame rotor position $\mathbf{r}_{b,i}$ and thrust T_i,

$$\tau = \sum_{i=1}^{N} \left(\mathbf{r}_{b,i} \times \mathbf{F}_{b,i} + \mathbf{a}_{b,i}\, \tau_i\right), \tag{21}$$

where τ_i is the signed shaft torque reported by the propulsion layer. In the current implementation this is the *aerodynamic load torque* Q_i from the prop model, signed by `rotor_dir[i]` (reaction torque on the body). Transient motor spin-up/spin-down effects enter separately through the rotor angular momentum terms $-\dot{\mathbf{H}}$ and $-\omega_b \times \mathbf{H}$; they are not double-counted inside τ_i.

The rigid-body angular dynamics include angular damping and the additional angular momentum stored in spinning rotors. Let $\mathbf{I}$ be the body inertia, ω_b the body rates, and define the rotor angular momentum and its derivative

$$\mathbf{H} = \sum_{i=1}^{N} J_i\, s_i\, \Omega_i\, \mathbf{a}_{b,i}, \qquad \dot{\mathbf{H}} = \sum_{i=1}^{N} J_i\, s_i\, \dot{\Omega}_i\, \mathbf{a}_{b,i}, \tag{22}$$

where J_i is the axial inertia of rotor i, Ω_i its spin rate (rad/s), and $s_i = -\texttt{rotor\_dir}[i]$ is the rotor spin sign (opposite the reaction-torque sign). The implemented equation of motion is

$$\mathbf{I}\, \dot{\omega}_b = (\tau - \mathbf{d} \odot \omega_b) - \omega_b \times (\mathbf{I}\omega_b) - \omega_b \times \mathbf{H} - \dot{\mathbf{H}}, \tag{23}$$

with elementwise angular damping $\mathbf{d}$ from `QuadrotorParams.angular_damping`. Quaternion kinematics use section 5.

7.3 Defaults and caveats

Defaults. `QuadrotorParams` provides constructor defaults (used if a spec omits values): mass 1.5 kg, diagonal inertia $(0.029125, 0.029125, 0.055225)\,\mathrm{kg\,m^2}$, linear drag $0.05\,\mathrm{N/(m/s)}$, and angular damping $(0.02, 0.02, 0.01)$.

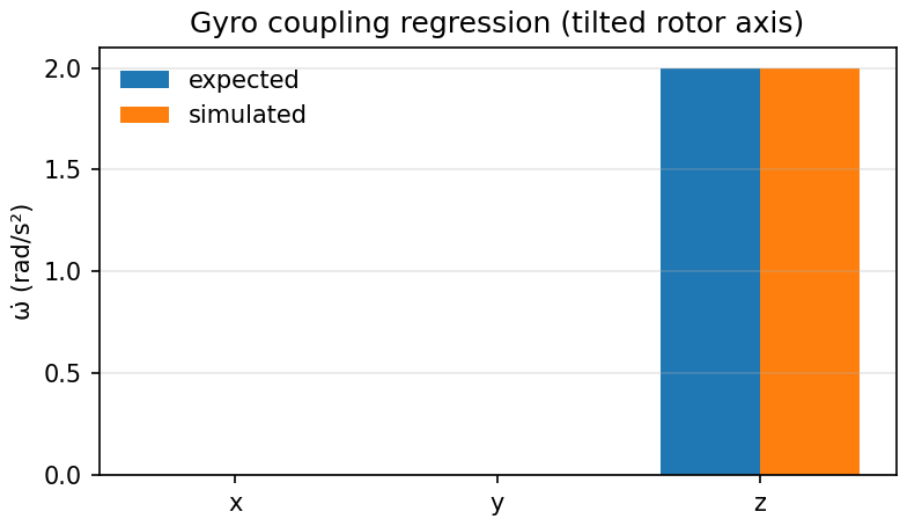

Figure 6: Gyro coupling regression for a tilted rotor axis: expected vs simulated $\dot{\omega}$ components.

Contact forces. The rigid-body model itself does not include ground contact. Contact is handled in the coupled plant layer: penalty contact adds an external COM force in the RHS, while constraint contact can use hybrid interval stepping (time-of-impact + impact map + grounded impulses). See section 10.4.

Aerodynamics. The drag term is deliberately simplistic (linear, isotropic, no lift/induced effects). This is appropriate for controller-in-the-loop regression but not for aerodynamic performance prediction.

7.4 TOML configuration: airframe geometry and initial state

The rigid-body parameters in section 7 are configured under `[airframe]` (`AirframeSpec` in `src/sim/Aircraft/Spec.jl`) and its nested `[airframe.x0]` initial-condition table. A typical multirotor configuration (matching the shipped Iris asset) is:

```
[airframe]
kind = "multirotor"
mass_kg = 1.5
inertia_diag_kgm2 = [0.029125, 0.029125, 0.055225]
# Optional products of inertia Ixy, Ixz, Iyz (defaults to zeros)
inertia_products_kgm2 = [0.0, 0.0, 0.0]

rotor_pos_body_m = [
  [ 0.1515, 0.2450, 0.0],
  [-0.1515, -0.1875, 0.0],
  [ 0.1515, -0.2450, 0.0],
  [-0.1515, 0.1875, 0.0],
]
rotor_axis_body_m = [
  [0.0, 0.0, 1.0],
  [0.0, 0.0, 1.0],
  [0.0, 0.0, 1.0],
  [0.0, 0.0, 1.0],
```

```
]

linear_drag = 0.05
angular_damping = [0.02, 0.02, 0.01]

[airframe.x0]
pos_ned = [0.0, 0.0, 0.0]
vel_ned = [0.0, 0.0, 0.0]
q_bn = [1.0, 0.0, 0.0, 0.0] # or euler_deg / euler_rad
omega_body = [0.0, 0.0, 0.0]
```

Parameter mapping to the dynamics. The keys above map one-to-one into the symbols used in section 7:

- `mass_kg` $\to m$.
- `inertia_diag_kgm2` and `inertia_products_kgm2` $\to$ the symmetric inertia tensor $\mathbf{I}$. (Alternatively, `inertia_kgm2` may be supplied as a full symmetric 3×3 tensor.)
- `rotor_pos_body_m` $\to \mathbf{r}_{b,i}$ for each rotor.
- `rotor_axis_body_m` $\to \mathbf{a}_{b,i}$ for each rotor. Axes are normalized during build; if omitted entirely, all axes default to $(0, 0, 1)$ (thrust along $-z_b$).
- `linear_drag` $\to c_d$ in $\mathbf{F}_{drag} = -c_d\, \mathbf{v}_{rel}$.
- `angular_damping` $\to \mathbf{d}$ in $\mathbf{d} \odot \omega_b$.

Initial condition semantics. The `[airframe.x0]` table sets the initial rigid-body state. Attitude may be specified either as `q_bn` or as Euler angles; in strict mode, only one representation may be supplied.

8 Actuation and propulsion

The canonical multirotor plant integrates actuator dynamics and propulsion dynamics inside one RHS (`Sim.PlantModels.CoupledMultirotorModel`) so that both fixed-step and adaptive integrators see a single continuous system.

8.1 Actuator channel mapping and dynamics

PX4 produces actuator commands as fixed-size ABI arrays: 12 motor channels in $[0, 1]$ and 8 servo channels in $[-1, 1]$ (`Vehicles.ActuatorCommand`). These are *normalized lockstep ABI units* (not PWM) taken from the uORB outputs subscribed by the bridge (`actuator_motors` and `actuator_servos`); they are interpreted as normalized ESC duty and control-surface deflection. The plant stores actuator outputs in the same fixed-size arrays ($\mathbf{y}_m \in \mathbb{R}^{12}$, $\mathbf{y}_s \in \mathbb{R}^8$), but the physical vehicle may have $N \leq 12$ propulsors.

Mapping from PX4 channels to physical motors. A typed mapping (`Vehicles.MotorMap\{N\}`) selects which PX4 motor channel drives each physical motor:

$$d_i \coloneqq (\mathbf{y}_m)[\texttt{motor\_channel}[i]], \qquad i \in \{1, \ldots, N\}. \tag{24}$$

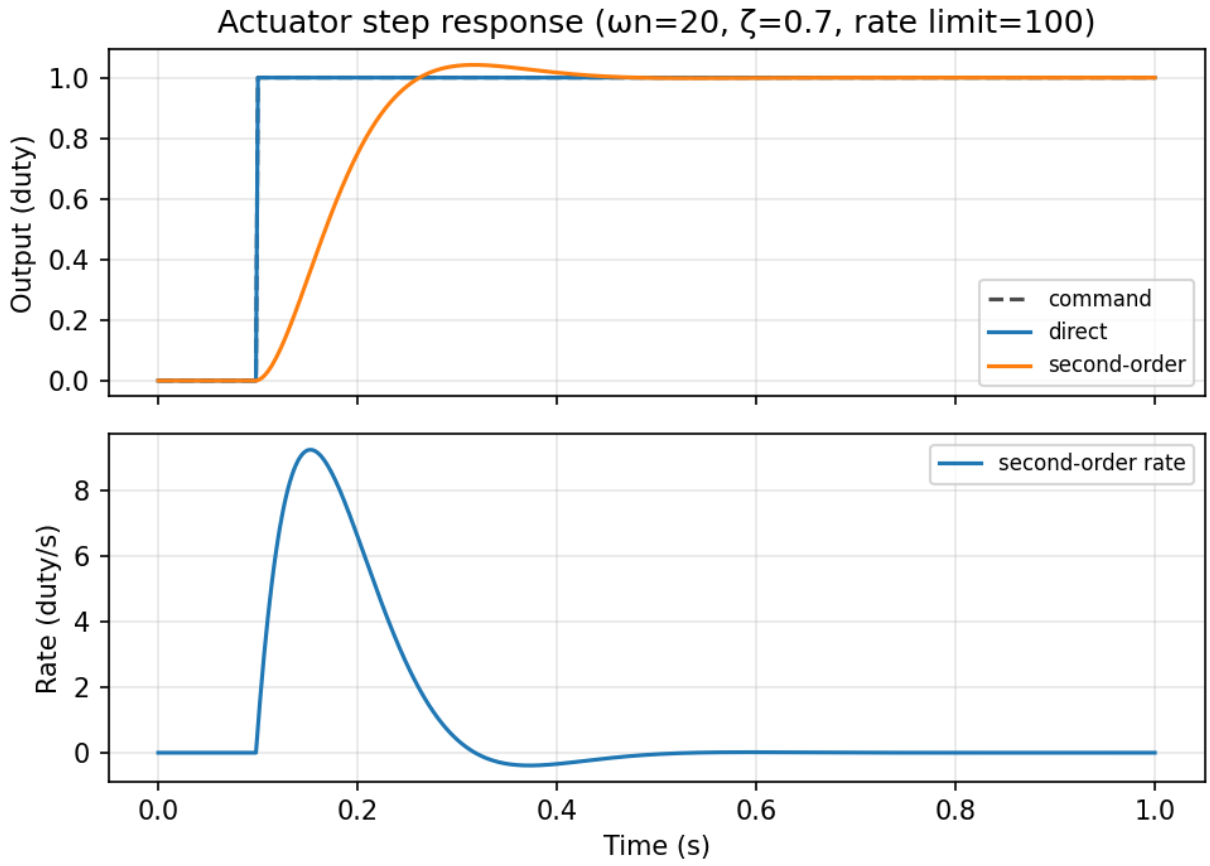

Figure 7: Actuator step response example: direct vs second-order dynamics with a rate limit.

Faults are defined over *physical* motors (indices $1..N$), not PX4 channels. The plant applies the motor-disable mask by forcing the affected physical duties to zero before evaluating propulsion.

Actuator dynamics. Actuator channel dynamics are integrated as part of the plant state. Motor and servo channels can each use one of the following models (`src/sim/Vehicles.jl`, `src/sim/PlantModels/CoupledMultirotor.jl`):

- **Direct**: algebraic output, snapped to the new command at each autopilot boundary via `plant_on_autopilot_tick`, then held.
- **First-order**: $\dot{\mathbf{y}} = (\mathbf{u}_{cmd} - \mathbf{y})/\tau$.
- **Second-order**: $\dot{\mathbf{y}} = \dot{\mathbf{y}}_{\text{eff}}$, $\ddot{\mathbf{y}} = \omega_n^2(\mathbf{u}_{cmd} - \mathbf{y}) - 2\zeta\omega_n\dot{\mathbf{y}}_{\text{eff}}$, with an optional rate limit implemented by clamping $\dot{\mathbf{y}}_{\text{eff}}$ in the RHS and projecting stored rate states post-step.

After each engine interval the coupled plant implements `plant_project` to clamp actuator outputs into ABI ranges and to project second-order rate states back into the configured rate limit, ensuring hard bounds are enforced deterministically.

8.2 Motor/ESC/prop model

Each physical motor uses a parameter-only container `Propulsion.MotorPropUnit` consisting of: (i) an ESC stage with efficiency η and deadzone, (ii) a quasi-static BLDC electrical model, and (iii) a quadratic propeller model with an inflow correction. The plant integrates rotor speed Ω_i (rad/s) for each physical motor.

ESC stage. The commanded duty is clamped to $[0, 1]$ and zeroed if it is below the ESC deadzone. The motor terminal voltage is

$$V_m = d\, V_{bus}. \tag{25}$$

BLDC electrical model (quasi-static current). Let K_v be specified in RPM/V. The back-EMF constant is

$$K_e = \frac{60}{2\pi K_v} \quad [\mathrm{V\,s/rad}], \tag{26}$$

and the torque constant is set equal to $K_t \equiv K_e$ (SI). Motor current is computed from applied voltage and back-EMF and then clamped:

$$I_m = \mathrm{clamp}\left(\frac{V_m - K_e\,\Omega}{R},\ 0,\ I_{\max}\right). \tag{27}$$

The electrical torque subtracts an approximate no-load current I_0:

$$\tau_e = K_t\, \max(0, I_m - I_0). \tag{28}$$

Loss modeling note. In this model, I_0 represents *electrical* no-load current (iron/core loss) that does not produce torque, while $b\,\Omega$ is a *mechanical* drag torque. The defaults are a pragmatic generic fit rather than a motor identification; if you want a single loss channel, set either $I_0 = 0$ (pure mechanical drag) or $b = 0$ (pure electrical loss) to avoid double-counting.

Quadratic propeller aerodynamics. At air density ρ the thrust and load torque are

$$T = k_T\, \rho\, \Omega^2\, f_T(\mu), \qquad Q = k_Q\, \rho\, \Omega^2\, f_Q(\mu), \tag{29}$$

with a simple inflow sensitivity factor

$$f(\mu) = \frac{1}{1 + k\,\mu^2}, \qquad \mu = \frac{V_{ax}}{\max(10^{-3}, |\Omega| R_{prop})}. \tag{30}$$

The prop radius R_{prop} is stored in the prop parameters. The coupled plant computes the axial component V_{ax} by projecting the *air velocity relative to the vehicle* onto each propulsor axis (section 5.3):

$$\mathbf{v}_{air,n} = \mathbf{w}_{ned} - \mathbf{v}_{ned}, \qquad \mathbf{v}_{air,b} = \mathbf{R}_{bn}^{\mathsf{T}}\, \mathbf{v}_{air,n}, \qquad V_{ax,i} = -\mathbf{v}_{air,b} \cdot \mathbf{a}_{b,i}. \tag{31}$$

The inflow correction uses μ^2, so the sign of V_{ax} affects neither f_T nor f_Q. This $1/(1 + k\mu^2)$ attenuation is an ad-hoc shaping term (not a blade-element or momentum-theory model) chosen for stability and simplicity; it cannot represent windmilling, vortex-ring, or braking regimes. See section 18.

Rotor-speed dynamics. In the coupled plant, rotor speed is a continuous state integrated by the same ODE integrator as the rest of the plant. The RHS evaluates

$$\dot{\Omega} = \frac{\tau_e - Q - b\,\Omega}{J}, \tag{32}$$

where J is the axial inertia and b is a viscous friction coefficient, and where Q is the aerodynamic load torque evaluated at the *current* Ω (no internal Euler step inside the RHS). A guard prevents unphysical negative spin: if $\Omega \leq 0$ and $\dot{\Omega} < 0$, the derivative is set to zero.

Bus current and signed reaction torque. The current drawn from the bus is

$$I_{bus} = \frac{d\,I_m}{\max(10^{-6}, \eta)}. \tag{33}$$

The quadratic prop model returns a positive load torque magnitude $Q \geq 0$. The coupled plant applies a per-rotor sign `rotor_dir[i]` to generate the signed reaction torque on the body,

$$Q_{body,i} = \texttt{rotor\_dir}[i]\, Q_i, \tag{34}$$

and passes this signed value into the rigid-body dynamics.

8.3 Default propulsion parameterization and calibration

The helper `Propulsion.default_multirotor_set` provides a generic baseline motor+prop set. It calibrates the thrust coefficient k_T by solving for the steady-state speed at duty $d = 1$ under a quadratic load and then bisection-searching k_T so that the resulting thrust matches a target.

In the aircraft builder (`src/sim/Aircraft/Build.jl`), this target is derived from the airframe mass and gravity:

$$T_{hover,rotor} = \frac{m\,g}{N}, \qquad T_{target} = \texttt{thrust\_calibration\_mult} \cdot T_{hover,rotor}, \tag{35}$$

with `thrust_calibration_mult`=2.0 by default. The drag torque coefficient is tied to k_T through `km_m` via $k_Q = \texttt{km\_m}\, k_T$.

Caveat. This calibration is a pragmatic way to get reasonable thrust scaling for a "generic" vehicle. It is not an identification procedure for a specific airframe; if you care about realism beyond closed-loop controllability, replace the parameters with airframe-specific values.

8.4 TOML configuration: actuation mapping and propulsion parameters

Actuation and propulsion are configured in two places in the aircraft spec:

- `[actuation]` defines how PX4 ABI channels map to physical motors/servos and which actuator dynamics model is used.
- `[airframe.propulsion]` defines the motor/ESC/prop parameters used by the coupled plant (section 8.2).

A minimal Iris-like configuration is:

```
[actuation]

[[actuation.motors]]
id = "motor1"
channel = 1

[[actuation.motors]]
id = "motor2"
channel = 2

[[actuation.motors]]
id = "motor3"
channel = 3

[[actuation.motors]]
id = "motor4"
channel = 4

[actuation.motor_actuators]
kind = "direct" # direct | first_order | second_order

[actuation.servo_actuators]
kind = "direct"

[airframe.propulsion]
kind = "multirotor_default"
km_m = 0.05
V_nom = 12.0
rho_nom = 1.225
rotor_radius_m = 0.127
inflow_kT = 8.0
inflow_kQ = 8.0
thrust_calibration_mult = 2.0
rotor_dir = [1, 1, -1, -1]

[airframe.propulsion.esc]
eta = 0.98
deadzone = 0.02

[airframe.propulsion.motor]
kv_rpm_per_volt = 920.0
r_ohm = 0.25
j_kgm2 = 1.2e-5
i0_a = 0.6
viscous_friction_nm_per_rad_s = 2.0e-6
max_current_a = 60.0
```

Actuation mapping semantics. Each entry in `[[actuation.motors]]` creates a named physical motor ID and assigns it a PX4 channel index. During build, IDs are resolved into a `MotorMap\{N\}` so that `channel` implements the lookup in section 8:

$$d_i \leftarrow (\mathbf{y}_m)[\texttt{channel}_i].$$

Unknown IDs referenced elsewhere (e.g. by buses) are rejected at build time.

Actuator model parameters. The actuator model may be given as a string shorthand (`kind= "direct"` etc) or as a table with parameters. For example, a second-order motor actuator with a finite rate limit uses:

```
[actuation.motor_actuators]
kind = "second_order"
"\u03C9n" = 20.0 # omega_n
"\u03B6" = 0.7 # zeta
rate_limit = 100.0 # units: (duty)/s
```

These keys map directly to ω_n, ζ, and the optional $\dot{\mathbf{y}}$ clamp described in section 8.

Propulsion parameter mapping. The propulsion keys match the symbols used in section 8.2:

- `kv_rpm_per_volt` $\rightarrow K_v$ (and thus $K_e = 60/(2\pi K_v)$ and $K_t = K_e$).
- `r_ohm` $\rightarrow R$ in $I_m = (V_m - K_e\Omega)/R$.
- `i0_a` $\rightarrow I_0$ in $\tau_e = K_t \max(0, I_m - I_0)$.
- `max_current_a` $\rightarrow I_{\max}$ current clamp.
- `j_kgm2` and `viscous_friction_nm_per_rad_s` $\rightarrow J$ and b in $\dot{\Omega} = (\tau_e - Q - b\Omega)/J$.
- `eta` and `deadzone` configure the ESC stage (section 8.2).
- `rotor_radius_m` $\rightarrow R_{prop}$ in the inflow ratio μ.
- `inflow_kT`, `inflow_kQ` $\rightarrow k$ in $f(\mu) = 1/(1 + k\mu^2)$ for thrust and torque respectively.
- `km_m` ties k_Q to k_T through $k_Q =$ `km_m` k_T in the default prop model.
- `thrust_calibration_mult` sets the target thrust multiplier used during build-time calibration (section 8.3).
- `rotor_dir[i]` signs the reaction torque applied to the body, $Q_{body,i} =$ `rotor_dir`$[i]\, Q_i$. Rotor spin sign is the opposite ($s_i = -$`rotor_dir`$[i]$) for gyroscopic coupling in section 7.

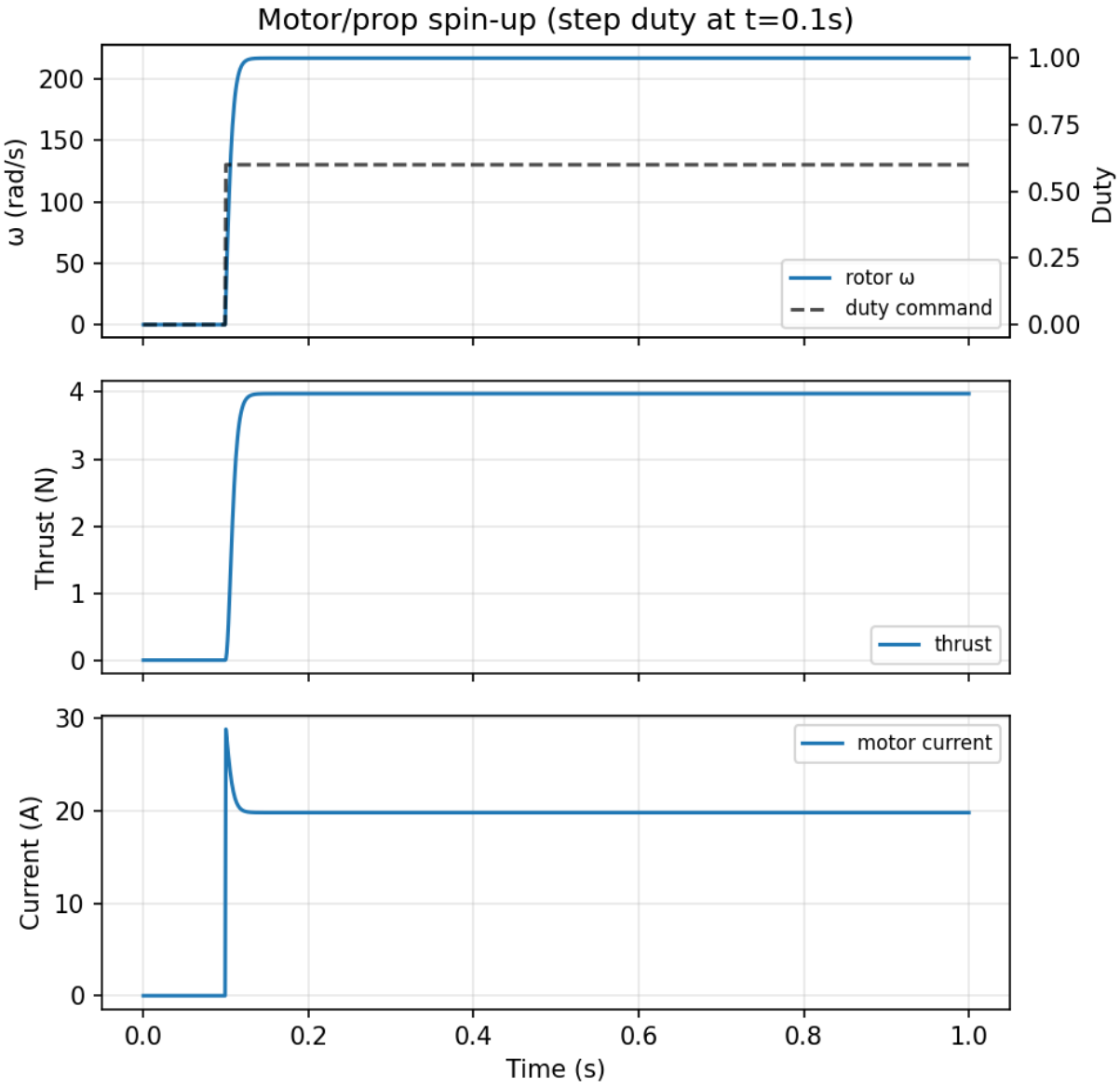

Figure 8: Motor/prop spin-up example for a step duty command at constant bus voltage.

9 Power network and battery models

The coupled multirotor plant solves battery-to-bus-to-motor coupling algebraically at each RHS evaluation and integrates battery internal states (SOC, polarization voltage, and optional temperature).

9.1 Battery state and model families

Battery state is part of `PlantState` as a `PowerState` with per-battery vectors:

$$\text{power} = \{\mathbf{s}, \mathbf{v}_1, \mathbf{T}\}, \tag{36}$$

where $\mathbf{s} \in [0,1]^B$ is state-of-charge (fraction remaining), $\mathbf{v}_1 \in \mathbb{R}^B$ is the Thevenin polarization voltage state, and $\mathbf{T} \in \mathbb{R}^B$ is battery temperature in °C.

Two battery model types are supported in the coupled plant (`src/sim/Powertrain.jl`, `src/sim/PlantModels/CoupledMultirotor.jl`):

- **IdealBattery**: constant open-circuit voltage, coulomb-counted SOC, no droop, no polarization state dynamics.
- **TheveninBattery**: open-circuit voltage $\mathrm{OCV}(s)$ via a table, series resistance $r_0(s,T)$ via a typed resistance model, and an optional polarization branch (r_1, C_1) with state v_1.

In both cases, the models are *parameter-only*: all dynamic state is kept inside `PlantState` so it is integrated by the engine.

9.2 Battery electrical dynamics

SOC is integrated from bus current by coulomb counting. For battery i with capacity C_i in coulombs and discharge current I_i:

$$\dot{s}_i = -\frac{\max(0, I_i)}{C_i}, \tag{37}$$

with a clamp that prevents SOC from decreasing below zero.

For `TheveninBattery` with a polarization branch (r_1, C_1), the internal voltage state evolves as

$$\dot{v}_{1,i} = -\frac{v_{1,i}}{r_1 C_1} + \frac{I_i}{C_1}, \tag{38}$$

and is disabled if $r_1 \le 0$ or $C_1 \le 0$.

9.3 Optional lumped thermal hook

Battery temperature is a plant state but is held constant unless the battery's `ThermalParams.enabled` flag is set. When enabled, the plant integrates a one-node thermal model:

$$\dot{T}_i = \frac{P_{loss,i} - k\,(T_i - T_{amb})}{C_{th}}, \tag{39}$$

where C_{th} is an effective thermal capacitance and k is a linear heat transfer coefficient to a fixed ambient temperature T_{amb}.

The internal electrical loss used by the code is

$$P_{loss} = I^2\, r_0(s,T) + \mathbf{1}_{r_1>0}\, \frac{v_1^2}{r_1}, \tag{40}$$

with the discharge-only current convention $I = \max(0, I_{bus})$.

9.4 Algebraic power network wiring

The plant supports multiple buses and multiple batteries through a small typed wiring object `PlantModels.PowerNetwork` (`src/sim/PlantModels/PowerNetwork.jl`). For N motors, B batteries, and K buses, the wiring specifies:

- `bus_for_motor[i]` $\in \{1, \ldots, K\}$
- `bus_for_battery[j]` $\in \{1, \ldots, K\}$
- `avionics_load_w[k]`: constant power load (W) on bus k
- `share_mode`: current sharing between batteries on a bus (`:inv_r0` or `:equal`).

The topology is intentionally simple: each battery belongs to exactly one bus.

9.5 Bus-voltage solve

For each bus k, the plant solves a scalar algebraic equation for $V_{bus,k}$ based on a Thevenin equivalent source and motor load currents.

Equivalent source for parallel batteries. For each battery i on bus k, define its instantaneous open-circuit terminal voltage

$$V_{0,i} = \mathrm{OCV}_i(s_i) - v_{1,i}, \tag{41}$$

and its series resistance $R_{0,i}(s_i, T_i)$. The plant forms a conductance-weighted average

$$w_i = \frac{1}{\max(R_{0,i}, R_\epsilon)}, \qquad V_{0,eq} = \frac{\sum_i w_i V_{0,i}}{\sum_i w_i}, \qquad R_{eq} = \begin{cases} 0, & \text{if all } R_{0,i} \le 0 \\ 1/\sum_i w_i, & \text{otherwise,} \end{cases} \tag{42}$$

with a small guard $R_\epsilon = 10^{-6}\,\Omega$ to avoid division by zero.

Avionics load approximation during the solve. Avionics draw is treated as constant power P_{av}. To keep the bus solve monotone and deterministic, the code approximates this term during the solve as a constant current computed at $V_{0,eq}$:

$$I_{av,assumed} \approx \frac{P_{av}}{V_{0,eq}}, \qquad V_{0,eff} = V_{0,eq} - R_{eq} I_{av,assumed}. \tag{43}$$

The motor load is then solved against the adjusted open-circuit voltage $V_{0,eff}$.

Fixed-point equation. Let $I_{motors}(V)$ be the total bus current drawn by all motors powered by this bus at bus voltage V (computed from the quasi-static motor current model, including deadzones and current limits). The bus voltage is defined as the clamped fixed point

$$V = \mathrm{clamp}_{[V_\ell, V_0]}\big(V_0 - R_{eq}\, I_{motors}(V)\big), \qquad V_\ell \equiv \min(V_{min}, V_0), \qquad V_0 \equiv V_{0,eff}, \tag{44}$$

where V_{min} is the maximum of all configured battery minimum voltages on the bus. If the open-circuit voltage drops below V_{min} (e.g. low SOC or cold temperature), the clamp interval collapses to $[V_0, V_0]$ and the solver returns $V = V_0$ rather than forcing an unphysical higher voltage.

Deterministic solver strategy. The implementation uses a staged deterministic approach (`src/sim/PlantModels/CoupledMultirotor.jl`):

1. A fast analytic solve assuming all active motors are in the unsaturated linear region, validated at the result.
2. A small fixed-count region-classified analytic iteration.
3. A fixed-iteration bisection on the monotone residual $g(V) = V + R_{eq}I(V) - V_0$.

All iteration counts are fixed to avoid data-dependent loop termination.

9.6 Per-bus and per-battery currents

After solving $V_{bus,k}$, the plant computes the true avionics current as $I_{av} = P_{av}/V_{bus,k}$ (guarded for $V \leq 0$) and forms the total per-bus current

$$I_{bus,k} = I_{motors,k} + I_{av,k}. \tag{45}$$

This current is then distributed across batteries on the same bus according to `share_mode`:

- `:equal`: equal current share.
- `:inv_r0`: share proportional to $1/r_0(s,T)$.

Battery disconnect fault. If `FaultState.battery_connected==false`, the plant forces all bus voltages to zero and all bus/battery currents to zero. This means the plant does not model residual motor windmilling or passive discharge under a disconnect fault; it is a hard "power removed" switch.

9.7 Battery telemetry for PX4

The plant implements the protocol `plant_outputs(model,t,x,u)` to expose algebraic outputs (`PlantOutputs` in `src/sim/Plant.jl`). This includes per-bus voltage/current and per-battery `BatteryStatus` records. The injected voltage is the solved bus voltage; the injected current is the bus current assigned to that battery by the sharing rule.

9.8 Series/parallel scaling and asset resolution

The power model supports configuring battery electrical parameters either at the *pack level* or at the *cell level* together with an explicit series/parallel topology. This is not just a documentation convention: the build system applies deterministic scaling rules that change the actual runtime parameters used by `Powertrain.TheveninBattery` (`src/sim/Aircraft/Build.jl`).

Pack topology. Each battery spec provides integers `series` $= N_s$ and `parallel` $= N_p$. These are used only for scaling; the runtime battery state remains a single Thevenin state per configured battery.

Voltage and cutoff scaling. If `ocv_is_cell=true`, the OCV curve is interpreted as a per-cell table $OCV_{cell}(s)$ and is scaled to pack voltage by

$$OCV_{pack}(s) = N_s\, OCV_{cell}(s). \tag{46}$$

Similarly, if `min_voltage_is_cell=true`, the hard cutoff voltage is scaled as $V_{\min,pack} = N_s V_{\min,cell}$.

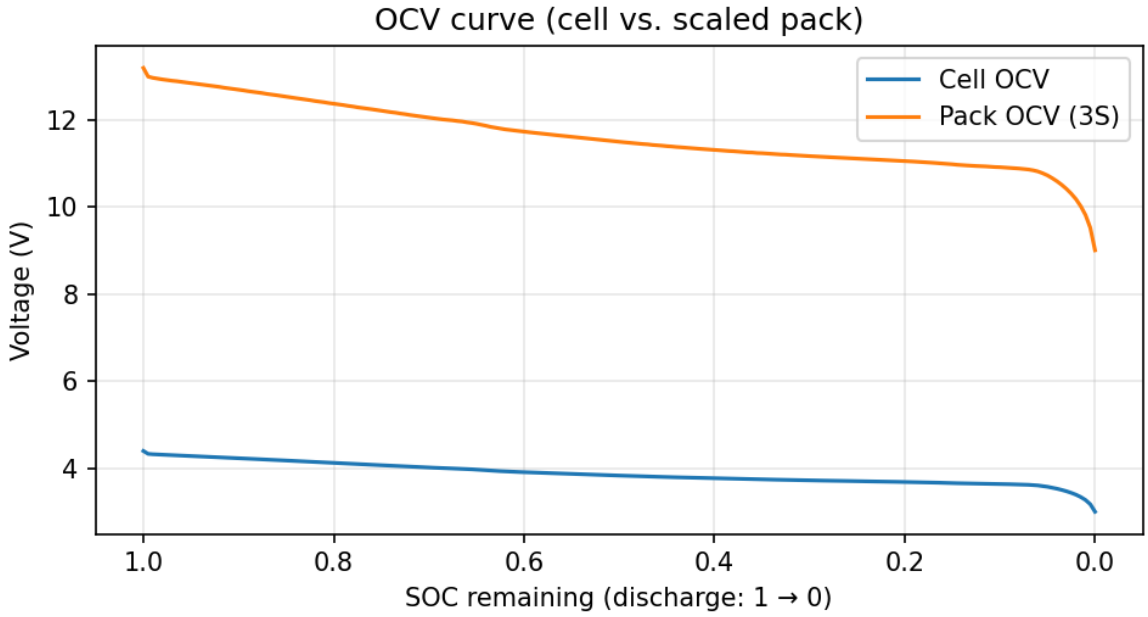

Figure 9: Cell OCV curve and scaled pack OCV for the example asset (3S shown).

Resistance and RC scaling. When the transient Thevenin parameters are marked cell-level via `thevenin_is_cell=true`, the builder applies

$$R_{0,pack} = \frac{N_s}{N_p} R_{0,cell}, \qquad R_{1,pack} = \frac{N_s}{N_p} R_{1,cell}, \tag{47}$$

$$C_{1,pack} = \frac{N_p}{N_s} C_{1,cell}, \qquad v_{1,pack}(0) = N_s \, v_{1,cell}(0), \tag{48}$$

which preserves the RC time constant $r_1 C_1$ under series/parallel aggregation while scaling voltages and resistances in the usual way. For an SOC/temperature resistance surface, the scaling is controlled independently by `r0_surface_is_cell` (defaults to `true` when the surface comes from a cell asset and `false` when it comes from a pack asset).

Pack-level overhead. `r0_overhead_ohm` is always interpreted as a *pack-level* additive term (connectors/wiring/BMS) and therefore is *not* affected by N_s, N_p.

Capacity. The state-of-charge integrator uses the pack capacity Q_{pack} (in ampere-hours). If `cell_capacity_ah` is provided and `capacity_ah` is not, then the builder sets

$$\texttt{capacity\_ah} = N_p \, \texttt{cell\_capacity\_ah}. \tag{49}$$

Otherwise, `capacity_ah` is used directly.

Battery assets. To reduce per-experiment boilerplate, a battery may reference a cell or pack `meta.toml` asset under `src/Workflows/assets/battery/`. Assets provide: (i) an OCV curve CSV with explicit SOC units/convention, (ii) optional resistance surfaces, (iii) optional thermal defaults. During build, asset-provided values are applied unless the corresponding TOML keys

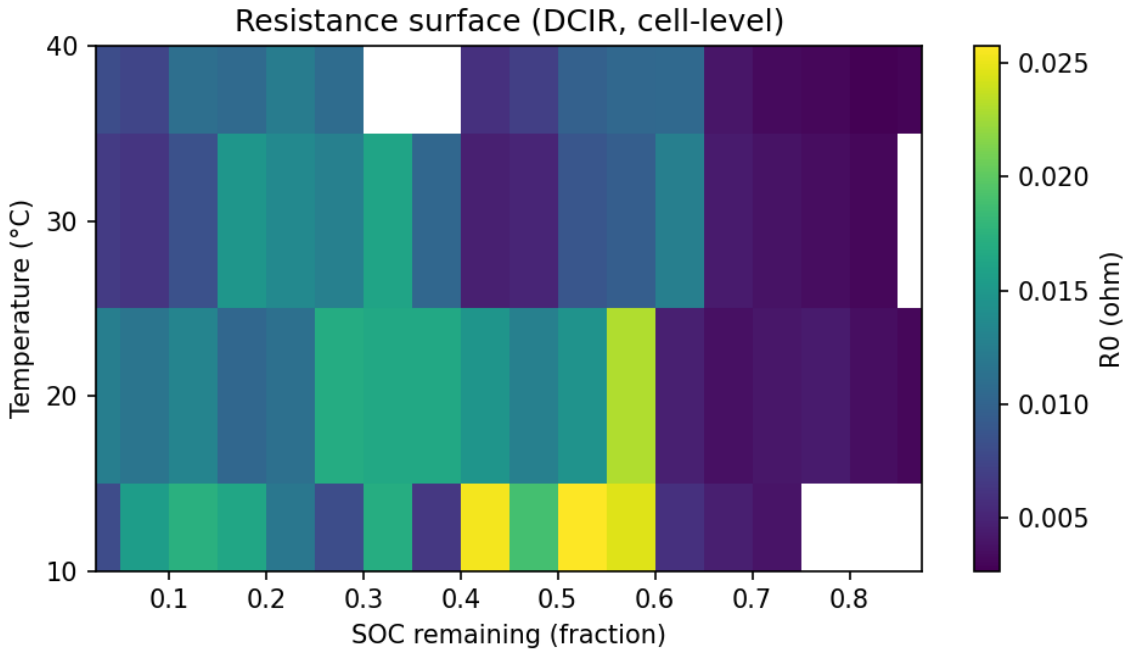

Figure 10: Example resistance surface (DCIR column, cell-level) as a function of SOC and temperature.

were explicitly set; explicit keys override asset values with a warning (see `_warn_override` in `src/sim/Aircraft/Build.jl`). The *TOML schema* currently enforces that a battery selects *either* `pack_asset` *or* `cell_asset` (not both), but a pack asset can internally reference its associated cell.

9.9 TOML configuration: batteries, buses, and sharing

The powertrain is configured in the `[power]` block (`PowerSpec` in `src/sim/Aircraft/Spec.jl`) with a list of batteries and buses.

Example 1: scalar-R0 Thevenin battery (Iris default).

```
[power]
share_mode = "inv_r0" # current sharing across parallel batteries on a bus

[[power.batteries]]
id = "bat1"
model = "thevenin"
capacity_ah = 5.0
soc0 = 1.0
ocv_soc = [0.0, 1.0]
ocv_v = [10.8, 12.6]
r0 = 0.020
r1 = 0.010
c1 = 2000.0
v1_0 = 0.0
min_voltage_v = 9.9
low_thr = 0.15
crit_thr = 0.10
emerg_thr = 0.05
```

```
[[power.buses]]
id = "main"
battery_ids = ["bat1"]
motor_ids = ["motor1","motor2","motor3","motor4"]
servo_ids = []
avionics_load_w = 0.0
```

Parameter mapping. For each battery entry:

- `id`: symbolic identifier used when wiring buses (`battery_ids`) and logging.
- `model`: `"thevenin"` (default) uses section 9.1; `"ideal"` uses a fixed `voltage_v` with SOC integration.
- `capacity_ah`: pack capacity Q_{pack} used in SOC integration.
- `soc0`: initial SOC $s(0)$ (fraction remaining).
- `ocv_soc`, `ocv_v`: piecewise-linear OCV table $OCV(s)$.
- `r0`, `r1`, `c1`, `v1_0`: Thevenin parameters in section 9.1; `r0` is used as a constant r_0 when no surface is provided.
- `min_voltage_v`: clamp floor $V_{\min}$ used by the bus solver when $V_0 \geq V_{\min}$; if $V_0 < V_{\min}$, the solver returns V_0 rather than forcing a higher voltage.
- `low_thr`, `crit_thr`, `emerg_thr`: SOC thresholds used only for the warning/telemetry state (section 9.1).

For each bus entry:

- `id`: bus name (logged and used for sanity checks).
- `battery_ids`, `motor_ids`, `servo_ids`: wiring lists. Each motor/battery must belong to exactly one bus.
- `avionics_load_w`: constant power draw $\bar{P}_k$ used in the bus load model (section 9.5).

Finally, `share_mode` selects the current-sharing rule among parallel batteries on a bus (section 9.6).

Example 2: cell asset + SOC/temperature R0(s,T) surface + thermal state. The following corresponds to `examples/specs/iris_example_3s2p_10ah_surface_rc_thermal_lockstep.toml` and demonstrates asset-backed configuration:

```
[[power.batteries]]
id = "bat1"
model = "thevenin"

# Reference a cell meta.toml; build a 3S2P pack (~12V, ~10Ah).
cell_asset = "../../src/Workflows/assets/battery/cells/example_3290mah_hv/meta.toml"
series = 3
parallel = 2
cell_capacity_ah = 5.0 # used to derive capacity_ah = parallel * cell_capacity_ah
soc0 = 1.0

# Use asset-provided resistance surface; pick which column becomes R0.
```

```
r0_surface_r0_col = "rdc_ohm"

# Optional transient RC branch (constant parameters).
r1 = 0.015
c1 = 120.0
v1_0 = 0.0
thevenin_is_cell = true

# Per-cell cutoff voltage, scaled by series.
min_voltage_v = 3.0
min_voltage_is_cell = true

[power.batteries.thermal]
enabled = true
ambient_temp_c = 25.0
initial_temp_c = 25.0
c_th_j_per_k = 600.0
k_to_ambient_w_per_k = 1.0
```

Additional parameter semantics.

- `cell_asset` / `pack_asset`: path to a `meta.toml` describing an asset; paths are resolved relative to the spec file (`_resolve_path` in `src/sim/Aircraft/TOMLIO.jl`).
- `series`, `parallel`: N_s, N_p used for the scaling rules in section 9.8.
- `cell_capacity_ah`: per-cell capacity; when `capacity_ah` is not explicitly set, the builder sets `capacity_ah` $= N_p$ `cell_capacity_ah`.
- `r0_surface_r0_col`: selects which CSV column becomes r_0 in the surface model (e.g., DCIR vs. ohmic). All other surface metadata (CSV path, SOC units/convention, temp/SOC column names) can be inherited from the asset or overridden explicitly via `r0_surface_csv_path`, `r0_surface_temp_col`, `r0_surface_soc_col`, `r0_surface_soc_units`, `r0_surface_soc_convention`.
- `thevenin_is_cell`: applies the RC scaling rules $(r_1, C_1, v_1(0))$ and, when using scalar `r0`, also scales r_0 by N_s/N_p. For a surface $r_0(s, T)$, use `r0_surface_is_cell` to control surface scaling.
- `thermal.*`: enables the optional temperature state integrated by the plant (see section 9.3). When enabled, the battery temperature drives $r_0(s, T)$ evaluation.

9.10 Battery asset schema and surface ingestion workflow

Battery assets live under `src/Workflows/assets/battery/` and are designed to be self-contained and version-controlled. At runtime, only the fields actually consumed by the loaders matter; additional metadata is permitted but ignored.

Cell asset (`cells/*/meta.toml`). A cell asset provides an OCV curve and (optionally) a resistance surface:

```
id = "example_3290mah_hv"
nominal_capacity_ah = 3.29
qualified_capacity_ah = 3.00

[ocv_curve]
csv = "ocv_soc.csv"
```

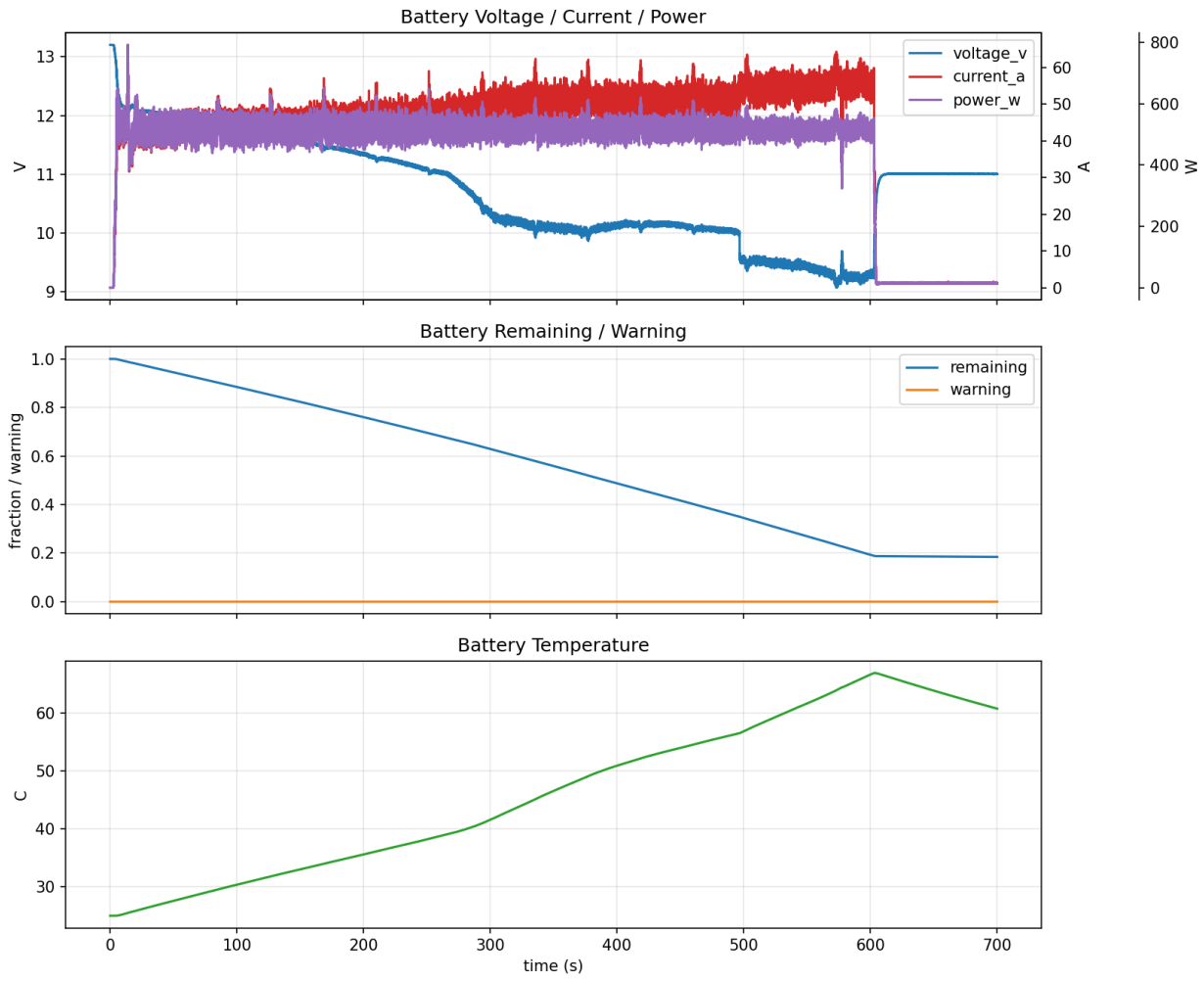

Figure 11: Battery runtime trace for the asset-backed example with thermal enabled (voltage, current, SOC, and temperature).

```
soc_col = "soc"
v_col = "ocv_v"
soc_units = "fraction" # or "percent"
soc_convention = "remaining" # or "used"

[resistance_surface_cell]
csv = "resistance_surface_cell.csv"
temp_col = "temp_c"
soc_col = "soc"
r0_col = "rdc_ohm" # column name selected by r0_surface_r0_col
soc_units = "fraction"
soc_convention = "remaining"
```

The builder loads these CSVs, normalizes SOC to "fraction remaining" in $[0, 1]$, and uses piecewise-linear (OCV) and bilinear (surface) interpolation with clamping to table bounds (`src/sim/Powertrain.jl`).

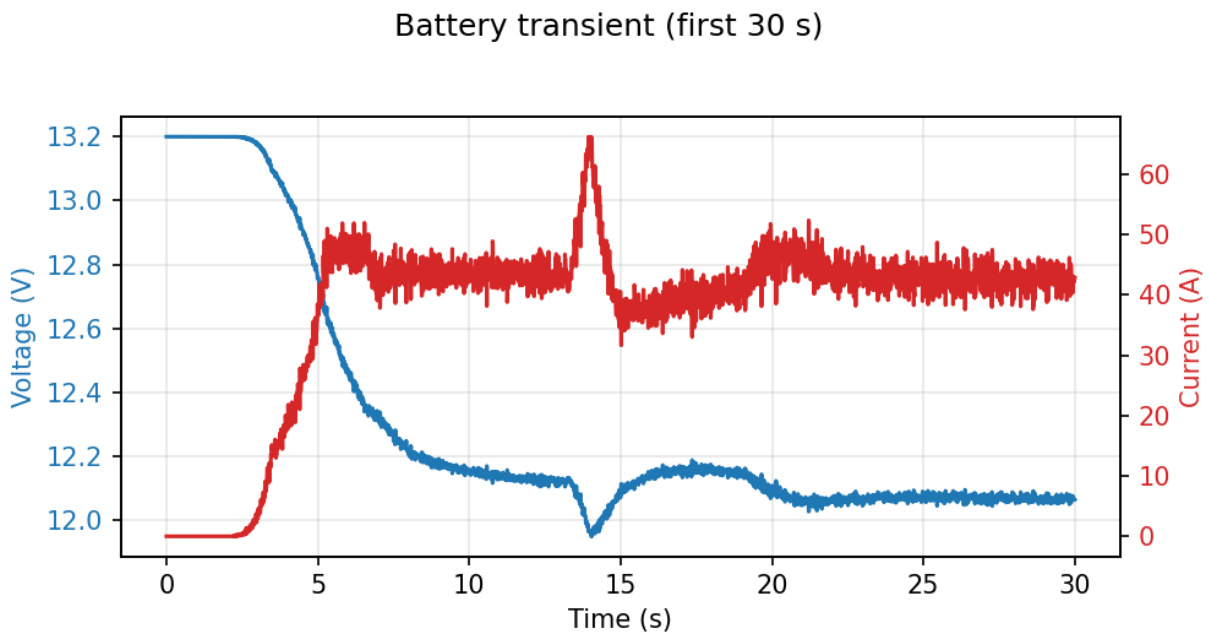

Figure 12: Battery voltage/current transient during the first 30 s of a lockstep mission run.

Pack asset (`packs/*/meta.toml`). A pack asset additionally specifies N_s, N_p and can provide a pack-level resistance surface and thermal defaults:

```
id = "example_8s1p_3290mah"
series = 8
parallel = 1

[cell]
ref = "../cells/example_3290mah_hv/meta.toml"

[resistance_surface_pack]
csv = "resistance_surface_pack.csv"
temp_col = "temp_c"
soc_col = "soc"
r0_col = "rdc_ohm"

[thermal]
c_th_j_per_k = 600.0
k_to_ambient_w_per_k = 1.0
```

Offline ingestion (how surfaces are generated). The repository includes an example ingestion script (`scripts/battery_assets/ingest_example_3290mah_8s1p.py`) that demonstrates one *heuristic* way to derive resistance surfaces from pulse traces. In that workflow, a DCIR-style

pulse at current I yields (pack-level) estimates

$$r_0^{(50\text{ms},100\text{ms})} \approx \frac{1}{2}\left(\frac{V_{pre} - V_{50\text{ms}}}{I} + \frac{V_{pre} - V_{100\text{ms}}}{I}\right), \tag{50}$$

$$r_{dc}^{(\text{end})} \approx \frac{V_{pre} - V_{min}}{I}, \tag{51}$$

$$r_1 \approx \max(r_{dc} - r_0, 0). \tag{52}$$

where V_{pre} is the pre-pulse voltage, $V_{0.05}$ and $V_{0.10}$ are the pulse voltages at 50 ms and 100 ms, and V_{min} is the minimum voltage during the pulse. These timing choices are *not canonical*; they are an example data-reduction choice to build an $r_0(s, T)$ surface for this simulator. If you have a different DCIR definition (e.g. HPPC windows), adjust the ingestion accordingly. The script then bins samples onto a (T, SOC) grid and writes `resistance_surface_*.csv`. This is an *offline* data-processing step; the simulator runtime only consumes the resulting CSV tables.

10 Environment and contacts

The environment is a composable container `Environment.EnvironmentModel` holding an atmosphere model, a wind model, a gravity model, and a world origin (`Types.WorldOrigin`). The runtime treats wind as a first-class sampled input on the bus to make record/replay exact.

10.1 Atmosphere: ISA 1976 (simplified)

The default atmosphere is `ISA1976` (`src/sim/Environment.jl`), providing temperature, pressure, and density as functions of MSL altitude. Altitude is computed from the NED position and the configured origin:

$$h_{msl} = h_{origin} - p_{ned,z}. \tag{53}$$

The ISA implementation clamps $h_{msl} \geq 0$ (no "below sea level" extrapolation).

10.2 Gravity

Gravity is supplied as an acceleration vector in NED (positive down). Two models are implemented (`src/sim/Environment.jl`):

- **UniformGravity**: constant $\mathbf{g}_{ned} = (0, 0, g)$.
- **SphericalGravity**: $g = \mu/r^2$ with $r \approx r_0 - p_{ned,z}$ (local-down approximation).

10.3 Wind

Wind is treated as a sampled input $\mathbf{w}_{ned}(t_k)$ published into the runtime bus at wind ticks (`src/sim/Sources/Wind.jl`). Between wind ticks the wind is held constant (sample-and-hold) and is consumed by both rigid-body drag and propulsion inflow calculations.

Ornstein–Uhlenbeck turbulence. The default wind model kind is `OUWind`, a 3-axis AR(1) gust process with a configurable correlation time τ and stationary standard deviation σ (`src/sim/Environment.jl`). This is a deliberate regression-friendly choice rather than a canonical

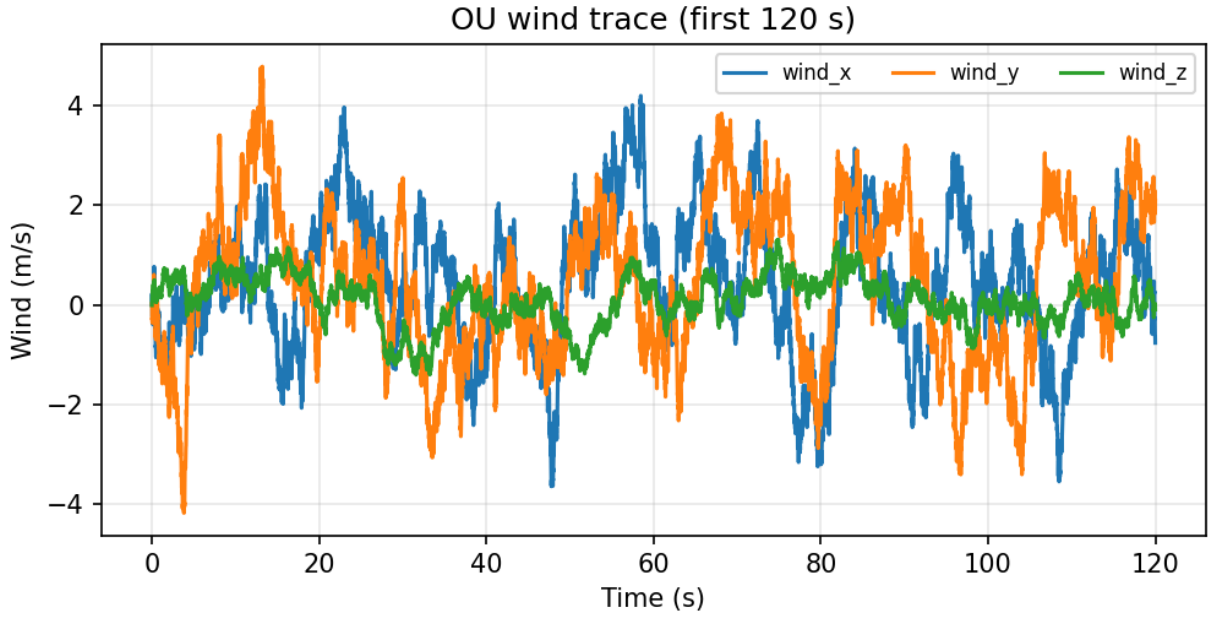

Figure 13: Example OU wind trace (first 120 s) from a lockstep run.

Dryden/Von Kármán spectrum; see section 18. The implementation uses the exact discrete-time update

$$\mathbf{v}_{gust}[k+1] = \phi\,\mathbf{v}_{gust}[k] + \left(\sigma\sqrt{1-\phi^2}\right) \odot \xi[k], \qquad \phi = e^{-\Delta t/\tau}, \quad \xi \sim \mathcal{N}(0, I). \tag{54}$$

The mean wind is then $\mathbf{w}_{ned} = \mathbf{w}_{mean} + \mathbf{v}_{gust}$, with optional additive step gusts supported for event injection.

Determinism. The wind source owns a dedicated RNG; all RNG usage occurs only inside its `update!` boundary method.

10.4 Contact model

Contact/terrain interaction is factored as an optional contact model (`src/sim/Contacts.jl`). The world frame is NED (down-positive) and the current terrain primitive is a flat ground plane at $z_{ned} = 0$. All current contact forces and impulses are applied at the vehicle center of mass (COM): contact produces translational forces/impulses but no moments (no landing-gear geometry, no multiple contact points, no terrain heightfields).

Ground primitive, gap function, and observables. For a rigid-body state $(\mathbf{p}_{ned}, \mathbf{v}_{ned})$ the implementation defines:

$$\phi(\mathbf{p}) := -p_{ned,z} \qquad \text{(gap; } \phi > 0 \text{ above the plane)}, \tag{55}$$

$$\delta(\mathbf{p}) := \max(0,\, -\phi) = \max(0,\, p_{ned,z}) \qquad \text{(penetration)}, \tag{56}$$

$$\mathbf{n}_{ned} := (0, 0, -1) \qquad \text{(unit normal; upward in NED)}. \tag{57}$$

These correspond directly to `ground_gap_m` and `ground_normal_ned` (`src/sim/Contacts.jl`). Tangential quantities are taken in the ground plane: $\mathbf{v}_t = (v_x, v_y)$, $v_t = \|\mathbf{v}_t\|$.

The contact layer exposes a lightweight `ContactInfo` snapshot (gap, penetration, mode, and instantaneous force estimates) through `PlantOutputs` for logging and for derived diagnostics (e.g. the physics-derived landed flag in `Runtime.Engine`). Modes are encoded as `CONTACT_AIRBORNE`, `CONTACT_IMPACTING`, and `CONTACT_GROUNDED`.

Two contact styles. Two contact styles are implemented and selectable via `PlantSpec.contact`:

- **Penalty-force model: `FlatGroundContact`.** A compliant spring–damper normal force with smooth Coulomb friction. This is evaluated directly in the plant RHS as an external force $\mathbf{F}_{contact,ned}$.
- **Constraint + hybrid model: `FlatGroundConstraintContact`.** Parameters for a unilateral (non-stiff) normal reaction for resting contact, plus deterministic hybrid handling in the coupled multirotor interval integrator: time-of-impact (TOI) localization + an impact map, and a fixed-step grounded time-stepping loop with an impulse-based contact solve.

10.4.1 Penalty-force model (FlatGroundContact)

The penalty model is active only when penetrating the plane ($\delta > 0$). The implemented normal force is

$$F_n = \max\big(0,\ k\,\delta + c\,\max(v_z, 0)\big), \tag{58}$$

where v_z is the NED vertical velocity (down-positive). The damping term resists compression only ($v_z > 0$). A smooth Coulomb friction approximation is applied in the tangential plane:

$$\mathbf{F}_t = -\mu F_n \,\frac{\mathbf{v}_t}{\|\mathbf{v}_t\| + v_\epsilon}. \tag{59}$$

The resulting NED contact force applied at the COM is

$$\mathbf{F}_{contact,ned} = (F_x, F_y, -F_n). \tag{60}$$

Friction can be disabled. Because the friction law is smooth, it does not model true stiction: as $\|\mathbf{v}_t\| \to 0$, $\mathbf{F}_t \to \mathbf{0}$.

10.4.2 Constraint + hybrid contact (FlatGroundConstraintContact)

The constraint contact path is implemented as a *hybrid* model: continuous-time forcing is used only for observables and for simple "force-only" integrations, while touchdown/bounce and resting contact are handled robustly via event localization and impulses inside `plant_integrate_interval` (`src/sim/PlantModels/CoupledMultirotor.jl`).

Grounding guard and slop band. A state is treated as a *ground candidate* when it lies on (or very near) the plane and is not moving upward beyond a small velocity tolerance:

$$\texttt{ground\_candidate}(x)\ :\ \ p_{ned,z} \geq -z_{\text{slop}} \ \ \wedge \ \ v_z \geq -v_{\text{slop}}. \tag{61}$$

The `z_slop_m` band is a numerical tolerance for treating small negative heights (slightly above the plane due to integration drift) as "on ground". The `vz_slop_mps` band treats tiny upward velocities as numerical noise; it reduces mode chattering when $v_z \approx 0$.

Continuous normal-reaction estimate (force-level evaluation). When `FlatGround\protect\penalty\z@ConstraintContact` is evaluated as a continuous force (e.g. in unit-test harnesses, or when computing `ContactInfo` from `plant_outputs`), it uses the *unconstrained* translational acceleration $\mathbf{a}_{ned}^{free}$ (gravity, thrust, drag, *no* contact) to compute a unilateral normal reaction:

$$F_n = \begin{cases} m\, a_z^{free}, & \text{if } \texttt{ground\_candidate}(x) \ \wedge \ a_z^{free} > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{62}$$

This cancels downward acceleration during resting contact (yielding $\ddot{z} \approx 0$) without introducing a stiff penalty spring. Continuous friction (when enabled) uses the same smoothed Coulomb form as section 10.4.1, but with this F_n.

Hybrid interval integration in the coupled multirotor plant. In the canonical runtime, the coupled multirotor plant overrides interval integration via the protocol hook `plant_integrate_interval` (section 2.4). The key design rule is: *integrate the smooth dynamics with `NoContact()` and apply contact impulses/maps separately.* In code, the interval integrator constructs

$$f_{\text{free}} \ := \ \texttt{CoupledMultirotorModel}(\ldots,\ \texttt{NoContact()},\ \ldots) \tag{63}$$

and uses f_{free} for lookahead, TOI localization, airborne stepping, and grounded substeps.

The interval advance is a small deterministic state machine:

1. If `ground_candidate` is true, advance with a grounded time-stepping substep of length $\Delta t = \min(\texttt{remaining}, \texttt{h\_contact\_us})$.
2. Otherwise, advance an airborne chunk using the configured integrator on f_{free}, but (when descending toward the ground) limit the chunk length to O(time-to-impact) so a crossing can be bracketed robustly.
3. If the airborne chunk brackets a descending crossing ($z_0 < 0$ and $z_1 \geq 0$), localize the TOI to integer microseconds by bisection and apply an impact map at that internal time.

The implementation caps the number of impacts handled inside a single outer interval at `MAX_IMPACTS_PER_INTERVAL=4` to avoid pathological "bounce storms".

Airborne guard step limiting (anti-tunneling guardrail). When above ground ($z < 0$) and closing ($v_z > 0$), the integrator estimates a time-to-impact

$$t_{\text{est}} = \frac{-z}{\max(v_z, v_\epsilon)}, \tag{64}$$

with $v_\epsilon = 1 \times 10^{-3}\,\text{m/s}$ and a safety factor $\gamma = 1.2$, and caps the next airborne integration chunk to

$$\Delta t_{\max} = \gamma\, t_{\text{est}}. \tag{65}$$

This does *not* guarantee a crossing is detected for highly non-monotone trajectories, but it greatly reduces deep tunneling when descending rapidly under coarse outer steps. (If no crossing is bracketed, no TOI localization is attempted.)

Deterministic TOI localization. If a lookahead step under f_{free} brackets a descending crossing, the code performs bisection on an integer microsecond interval $[0, \Delta t_{us}]$. Using a "probe" integrator reset to a clean state for each mid-point, it finds the smallest integer τ_{us} such that

$$z(t_0 + \tau_{us}) \geq 0, \tag{66}$$

terminating when the bracket width is $\leq 1\,\mu\text{s}$. The plant is then advanced to $t_0 + \tau$ under f_{free} to obtain the pre-impact state x^-.

Impact map (normal restitution + optional tangential friction). At the localized TOI, the rigid-body position is projected onto the plane ($p_z^+ = 0$). If the pre-impact normal velocity is closing ($v_z^- > 0$), the post-impact normal velocity is

$$v_z^+ = -e\, v_z^-, \tag{67}$$

where e is the configured restitution, with an inelastic "capture" rule:

$$e = 0 \quad \text{if} \quad |v_z^-| < v_{\text{rest}}. \tag{68}$$

This prevents micro-bounce/Zeno behavior when settling.

If `enable_impact_friction` is enabled, the impact map also applies a tangential impulse limited by a Coulomb bound. Let the tangential (XY) velocity vector be $\mathbf{v}_t^- = (v_x^-, v_y^-)$ with magnitude $v_t^- = \|\mathbf{v}_t^-\|$, and define the maximum tangential speed reduction

$$\Delta v_{t,\max} = \mu(1 + e)\, v_z^-. \tag{69}$$

The implemented update is

$$\mathbf{v}_t^+ = \begin{cases} \mathbf{0}, & v_t^- \leq \Delta v_{t,\max} \quad \text{(sticking impact)}, \\ \left(1 - \dfrac{\Delta v_{t,\max}}{v_t^-}\right) \mathbf{v}_t^-, & \text{otherwise.} \end{cases} \tag{70}$$

Attitude and body rates are unchanged by the current impact map (COM impulse only).

Grounded time-stepping loop. While `ground_candidate` remains true, the plant advances in fixed substeps of size `h_contact_us` (default $1000\,\mu\text{s}$). Each grounded substep uses a single evaluation of the *smooth* derivative $d = f_{\text{free}}(t, x, u)$ and then applies a closed-form, single-contact impulse solve.

Relation to contact time-stepping literature. This hybrid approach (TOI localization + impact map + fixed-step impulse updates) is in the family of impulse-based time-stepping and complementarity-inspired contact methods. It prioritizes determinism and stability over strict conservation; see the design rationale in section 18 and the limitations in section 19.

1) Smooth Euler update for non-contact states. For all non-rigid-body states (actuators, rotors, battery), the grounded substep uses explicit Euler:

$$y_{k+1} = y_k + \dot{y}_k\, \Delta t. \tag{71}$$

Angular rates are also advanced by Euler, and quaternions are renormalized after the Euler update.

2) Symplectic translation update with contact impulses. Let m be the vehicle mass and $\mathbf{a}_{ned}^{free}$ the unconstrained translational acceleration from d. The substep first predicts a free-flight velocity

$$\mathbf{v}^{pred} = \mathbf{v}_k + \mathbf{a}_{ned}^{free}\,\Delta t. \tag{72}$$

It then applies a velocity-level unilateral normal impulse:

$$\lambda_n = \begin{cases} m\,v_z^{pred}, & v_z^{pred} > 0, \\ 0, & v_z^{pred} \le 0, \end{cases} \qquad v_z^{post} = \begin{cases} 0, & v_z^{pred} > 0, \\ v_z^{pred}, & v_z^{pred} \le 0, \end{cases} \tag{73}$$

so downward (penetrating) velocity is cancelled, while upward velocity is left untouched (allowing liftoff).

If friction is enabled and $\lambda_n > 0$, a tangential Coulomb impulse with stiction is applied. With tangential velocity $\mathbf{v}_t^{pred} = (v_x^{pred}, v_y^{pred})$ and magnitude $v_t^{pred} = \|\mathbf{v}_t^{pred}\|$,

$$J_{t,\mathrm{req}} = m\,v_t^{pred}, \qquad J_{t,\max} = \mu\,\lambda_n. \tag{74}$$

The post-impulse tangential velocity is

$$\mathbf{v}_t^{post} = \begin{cases} \mathbf{0}, & J_{t,\mathrm{req}} \le J_{t,\max} \quad \text{(stiction)}, \\ \left(1 - \dfrac{J_{t,\max}}{m\,v_t^{pred}}\right)\mathbf{v}_t^{pred}, & \text{otherwise.} \end{cases} \tag{75}$$

Finally, the position update uses the post-impulse velocity (symplectic Euler for translation):

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \mathbf{v}^{post}\,\Delta t. \tag{76}$$

3) Position-only post-stabilization. To remove numerical drift without injecting momentum, the grounded loop snaps small height errors to the plane by clamping the next position:

$$p_{z,k+1} \leftarrow \begin{cases} 0, & p_{z,k+1} > 0, \\ 0, & |p_{z,k+1}| \le z_{\mathrm{slop}} \ \wedge\ v_z^{post} \ge 0, \\ p_{z,k+1}, & \text{otherwise.} \end{cases} \tag{77}$$

Velocities are intentionally *not* clamped in this post-stabilization.

4) Deterministic per-substep projection. At the end of each grounded substep the plant applies `plant_project` to enforce hard bounds (e.g. $\omega \ge 0$, SOC $\in [0, 1]$) during contact stepping as well as during free flight.

Impact telemetry. Hybrid impacts occur at an internal time τ inside a union-axis interval, so they do not align with log ticks. To preserve observability at low log rates, the interval integrator can return metadata:

- `impact_count`: number of impacts in the interval,
- `impact_dv_ned`: the maximum $\Delta\mathbf{v}$ (NED, m/s) among those impacts,
- `impact_time_us`: the microsecond timestamp of that max-Δv impact.

The runtime accumulates these into bus-level latched signals and additionally exposes a "1-tick equivalent" acceleration estimate

$$\mathbf{a}_{\text{impact\_est}} \approx \frac{\Delta \mathbf{v}}{\Delta t_{ap}}, \tag{78}$$

where Δt_{ap} is the nominal autopilot tick period (`timeline.dt_autopilot_s`). Because impacts are applied as discrete velocity jumps, the spike appears in `impact_accel_est` (and `impact_dv`) but not in the continuous acceleration or specific-force channels (`acc_x/y/z`, `spec_bx/by/bz`), which reflect the smooth dynamics evaluated at log boundaries.

Caveats (explicit in the current code).

- **COM contact only:** no moments, no attitude impulses, no rolling/tilting gear contact.
- **Single plane primitive:** no terrain slope, heightfield, or obstacle geometry.
- **Impulse/contact simplifications:** the grounded solver is a one-contact impulse update (a special-case closed-form of a frictional complementarity solve) and does not model compliance. Restitution is applied only in the discrete impact map; grounded stepping is effectively inelastic in the normal direction.
- **Quantization/guardrails:** TOI is localized to integer microseconds; the maximum number of impacts handled inside one outer interval is capped.

10.5 TOML configuration: environment and contact

Environmental forcing is configured under `[environment]` (`EnvironmentSpec` in `src/sim/Aircraft/Spec.jl`). Contact/terrain is configured under `[plant]` as `PlantSpec.contact`.

Environment.

```
[environment]
# Wind model ("none" | "constant" | "ou")
wind = "ou"
wind_mean_ned = [0.0, 0.0, 0.0] # m/s
wind_sigma_ned = [1.5, 1.5, 0.5] # m/s (OU only)
wind_tau_s = 3.0 # s (OU only)

# Atmosphere and gravity
atmosphere = "isa1976" # currently only "isa1976"
gravity = "uniform" # "uniform" | "spherical"
gravity_mps2 = 9.80665 # used only when gravity="uniform"
gravity_mu = 3.986004418e14 # used only when gravity="spherical"
gravity_r0_m = 6378137.0 # used only when gravity="spherical"
```

Parameter mapping.

- `wind`: selects `NoWind`, `ConstantWind`, or `OUWind` (section 10.3).
- `wind_mean_ned`: mean wind velocity $\bar{\mathbf{w}}$ in NED.
- `wind_sigma_ned` and `wind_tau_s`: OU parameters σ and τ. The implementation uses an exact discrete-time update (`OUWind.step_wind!`) rather than Euler–Maruyama.
- `atmosphere`: selects the atmosphere model (currently only `isa1976`).

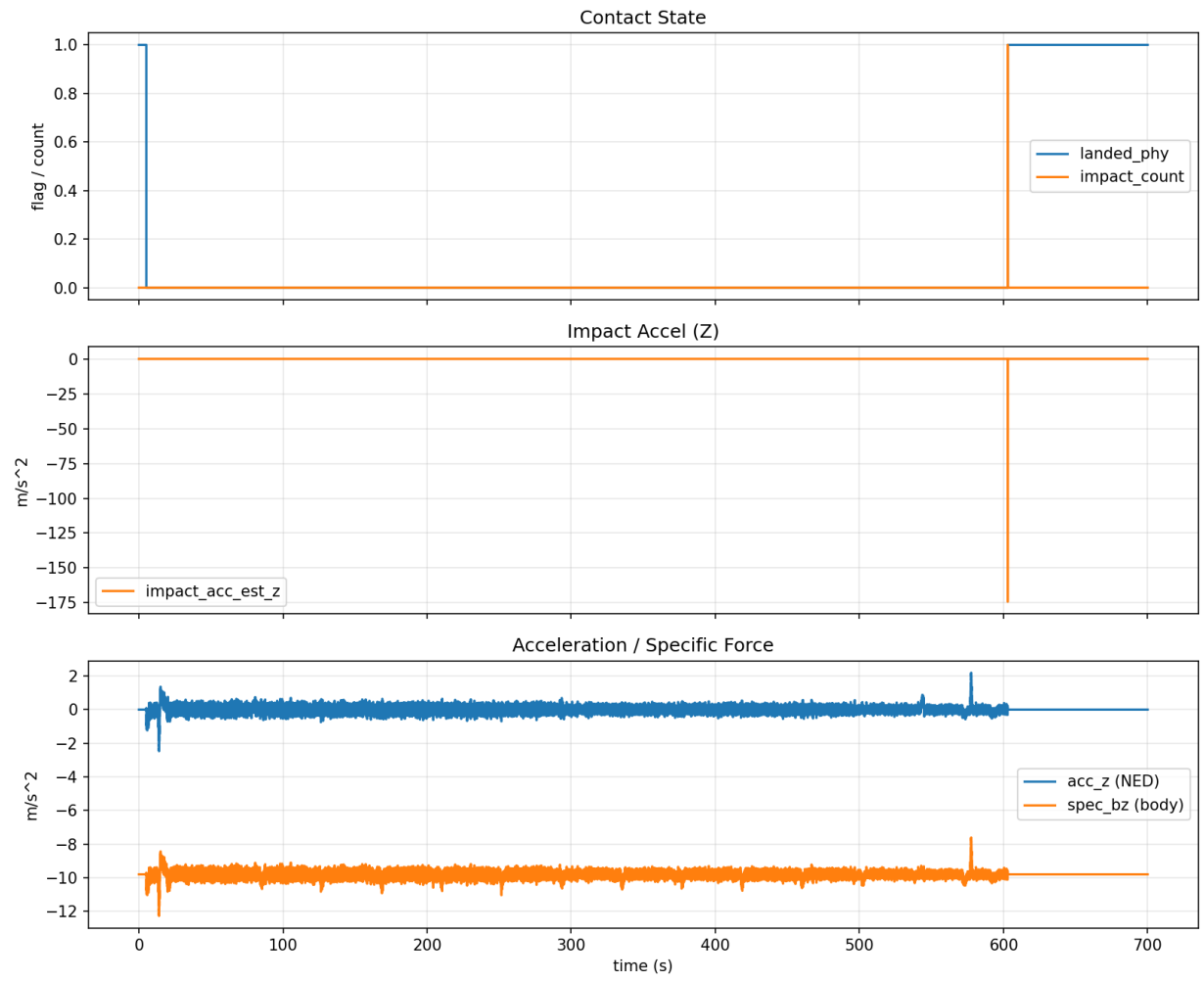

Figure 14: Contact trace from a long mission run (grounded flag, impact count, and impact acceleration proxy) around touchdown.

- `gravity`: selects uniform gravity or an inverse-square μ/r^2 model (`uniform` or `spherical`).
- `gravity_mps2`, `gravity_mu`, `gravity_r0_m`: parameters used by the selected gravity model.

Geodesy vs. gravity radii. The NED-to-LLA conversion uses the WGS84 equatorial radius (`EARTH_RADIUS_M` in `Autopilots.jl`); the default spherical gravity model now uses the same value for `gravity_r0_m`. Both remain independently configurable if a different convention is required.

Contact. The contact model is configured as either a shorthand string or a parameter table:

```
[plant]
# Shorthand strings:
contact = "flat_ground_constraint" # default
# contact = "flat_ground" # penalty spring-damper
# contact = "no_contact" # disable contact entirely

# Table form (FlatGroundConstraintContact)
```

```
contact = { kind="flat_ground_constraint",
  mu = 0.8,
  v_eps = 0.05,
  enable_friction = true,

  # Hybrid impact map (touchdown)
  restitution = 0.0, # e (0=inelas., 1=elas.)
  v_rest_threshold_mps = 0.05,
  enable_impact_friction = true,

  # Grounded time-stepping loop
  h_contact_us = 1000, # us substep while grounded
  z_slop_m = 1e-6,
  vz_slop_mps = 1e-3
}
```

Contact parameter mapping.

- **Penalty model (`FlatGroundContact`).** Keys map to the compliant spring–damper:
  - `k_n_per_m`: normal spring constant k in $F_n = k\,\delta + c\,v_z$.
  - `c_n_per_mps`: normal damping c (applied only when compressing, $v_z > 0$).
  - `mu` (also accepts unicode μ): friction coefficient μ in the smoothed Coulomb model.
  - `v_eps`: smoothing velocity v_ϵ.
  - `enable_friction`: toggles friction; the normal spring–damper remains active.
- **Constraint + hybrid model (`FlatGroundConstraintContact`).** Keys map to the guard, friction, and hybrid layers:
  - *Grounding guard / slop band:* `z_slop_m` (vertical tolerance treated as "at ground").
  - *Velocity slop:* `vz_slop_mps` (treats small upward v_z as numerical noise in the ground candidate predicate).
  - *Friction parameters:* `mu` (also accepts unicode μ) and `enable_friction` are used both by the (optional) continuous force-level friction estimate and by the Coulomb bounds in the impulse-based impact map / grounded solve. The smoothing velocity `v_eps` affects only the continuous force-level estimate.
  - *Hybrid impact map (touchdown/bounce):* `restitution` (alias `e`), `v_rest_threshold_mps`, and `enable_impact_friction` (toggles only the tangential impulse applied at impact).
  - *Grounded time-stepping:* `h_contact_us` (fixed microsecond substep size while grounded).

11 Estimator and scenario layers

PX4 expects "estimated" state topics even when a full sensor/EKF loop is not simulated. `PX4Lockstep.Sim` therefore inserts two discrete-time layers between truth and PX4:

1. an **estimator model** that maps truth state to an estimated state (optionally noisy/delayed), and
2. a **scenario model** that provides high-level autopilot commands and fault injections.

Both layers publish to the runtime bus at their own boundaries and are treated as sample-and-hold between boundaries.

11.1 Estimated state

The estimated state packet used by the autopilot source is

$$\hat{x} \coloneqq \{\hat{\mathbf{p}}_{ned},\ \hat{\mathbf{v}}_{ned},\ \hat{\mathbf{q}}_{\hat{bn}},\ \hat{\omega}_{body}\} \tag{79}$$

(`Estimators.EstimatedState` in `src/sim/Estimators.jl`).

11.2 Noisy estimator with AR(1) biases

`NoisyEstimator` injects (i) additive Gaussian noise and (ii) slowly-varying biases modeled as AR(1) processes (implemented as discretized Ornstein–Uhlenbeck):

$$b[k{+}1] = \phi\, b[k] + \sigma\, \sqrt{1-\phi^2}\, \xi[k], \qquad \phi = e^{-\Delta t/\tau},\ \xi \sim \mathcal{N}(0,1). \tag{80}$$

Biases are applied to position, velocity, yaw, and body rates; noise is applied additively. Yaw noise/bias is applied as a rotation about the NED $+z$ axis by left-multiplying the truth quaternion with an axis-angle quaternion. Because the multiplication is on the left, the yaw error is expressed in the inertial/NED frame (not body frame).

Deterministic stepping cadence. The estimator API receives an explicit `dt_hint` from the engine. The noisy estimator uses `dt=dt_hint` (rather than recomputing `t-last_t`) to avoid float subtraction drift.

11.3 Quantized fixed delay

`DelayedEstimator` wraps an inner estimator and applies a fixed delay implemented as a step-quantized ring buffer. Both the configured `dt_est` and `delay_s` are required to be exact integer microseconds; additionally, `delay_s` must be an exact multiple of `dt_est`. At runtime, the wrapper validates that the engine's provided `dt_hint` matches `dt_est` exactly; otherwise it errors.

11.4 Scenario outputs and faults

Scenarios publish a small `ScenarioOutput` packet into the bus (`src/sim/Scenario.jl`):

$$y_{scn} \coloneqq \{\texttt{ap\_cmd},\ \texttt{landed},\ \texttt{faults},\ \mathbf{w}^{\Delta}_{ned}\}. \tag{81}$$

The high-level command `AutopilotCommand` includes (armed, mission request, RTL request). `FaultState` is a compact immutable mask containing:

- a motor disable bitmask over physical motor indices (up to 32),
- a battery connected flag, and
- a generic sensor/estimator fault bitmask.

Faults are treated as sample-and-hold between boundaries and are consumed by the plant and (optionally) by other sources.

Caveat. The physics-derived landed flag is not yet consistently propagated through the PX4 navigation stack at touchdown; the navigator may not observe the landing transition even when the contact model reports a grounded state. This is a known limitation to be addressed.

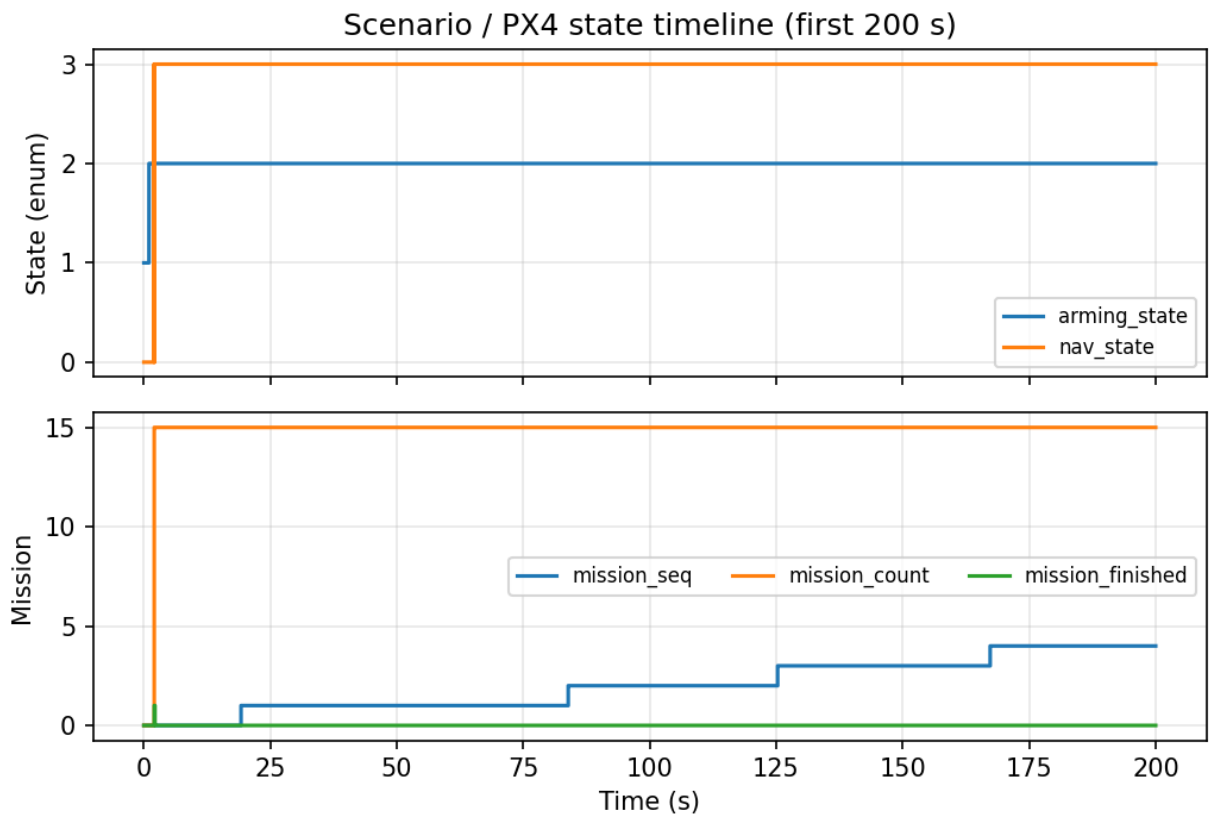

Figure 15: Scenario/PX4 state timeline excerpt (arming state, navigation state, and mission sequence) from a long mission run.

11.5 Scripted and event-driven scenarios

Two built-in scenario types exist:

- `ScriptedScenario`: time-based arming and mission start with a simple landed detector and an optional "force takeoff" override.
- `EventScenario`: a small deterministic event scheduler supporting `AtTime` and `AtStep` events for arming, mission/RTL pulses, wind steps, motor/battery faults, and threshold-triggered actions (e.g. "when SOC below ...").

For event-driven scenarios that carry a static scheduler, the engine can discover `AtTime` boundaries up front and include them as true event-axis times (`Sources.Scenario.event_times_us`).

11.6 TOML configuration: estimator and scenario scheduling

Scenario timing and estimator behavior are configurable through `[scenario]` (`ScenarioSpec`) and `[estimator]` (`EstimatorSpec`):

```
[scenario]
arm_time_s = 1.0
mission_time_s = 2.0
```

```
[estimator]
kind = "noisy_delayed" # "noisy_delayed" | "none"
pos_sigma_m = [0.02, 0.02, 0.02]
vel_sigma_mps = [0.05, 0.05, 0.05]
yaw_sigma_rad = 0.01
rate_sigma_rad_s = [0.005, 0.005, 0.005]
bias_tau_s = 50.0
rate_bias_sigma_rad_s = [0.001, 0.001, 0.001]
delay_s = 0.02 # seconds; set to null for zero delay
dt_est_s = 0.004 # seconds; set to null to publish at autopilot ticks
```

Scenario parameters.

- `arm_time_s`: time at which the scenario requests arming (fed into `AutopilotCommand` and forwarded via `set_cmd!`).
- `mission_time_s`: time at which mission start is requested (mapped to commander-lite mission requests in the C runtime).

Estimator parameter mapping. The noise and bias parameters directly configure the stochastic terms in the estimator equations above:

- `pos_sigma_m`, `vel_sigma_mps`, `yaw_sigma_rad`, `rate_sigma_rad_s`: measurement noise standard deviations.
- `bias_tau_s` and `rate_bias_sigma_rad_s`: OU bias parameters for the gyro-rate bias process.
- `delay_s`: sample-and-hold transport delay; the estimator output at time t is based on plant truth at $t -$ `delay_s` (quantized to the microsecond timeline).
- `dt_est_s`: estimator publication cadence. If `null`, estimator publications are aligned to the autopilot axis.

12 PX4 lockstep integration

PX4 is treated as a deterministic discrete-time component stepped by integer microseconds. The simulation engine never advances PX4 by wall-clock time; instead it calls the lockstep step function with an exact `UInt64` timestamp at each autopilot boundary.

12.1 Autopilot step interface

The canonical interface is `Autopilots.autopilot_step` (`src/sim/Autopilots.jl`). On each autopilot tick, the engine provides:

- estimated state: $(\hat{\mathbf{p}}_{ned}, \hat{\mathbf{v}}_{ned}, \hat{\mathbf{q}}_{bn}, \hat{\omega}_{body})$,
- high-level command `AutopilotCommand` (arm/mission/RTL),
- a best-effort `landed` flag, and
- per-battery `BatteryStatus` records computed by the plant outputs protocol.

State-to-global conversion. PX4 expects global coordinates for some topics. The implementation converts local NED to an approximate latitude/longitude/altitude using a fixed Earth radius and the configured home origin (`_ned_to_lla`). Altitude is computed as $h_{msl} = h_{home} - p_{ned,z}$. This approximation is intended for local missions (order $\sim$10 km from home) and uses a fixed $\cos(\varphi_0)$ longitude scale; see section 18.

Mode selection. The Julia bridge forwards the high-level command packet to the C runtime (`set_cmd!`); when Commander is disabled, the lockstep C runtime publishes the minimal commander-lite topics (`vehicle_status`, `vehicle_control_mode`, `actuator_armed`) and drives `nav_state`/`arming_state` based on command inputs and `mission_result`. A configurable `edge_trigger` mode determines whether mission/RTL requests are treated as edges or levels.

12.2 Multi-rate uORB injection

PX4Lockstep uses explicit injection sources to publish uORB messages at configurable integer-microsecond periods (`src/sim/Autopilots/UORBInjection.jl`). The boundary contract is:

1. queue all uORB publishes that are *due* at the current `time_us`,
2. call `step_uorb!(handle,time_us)` exactly once, then
3. read back configured uORB subscriptions.

The default injector publishes a set of "state injection" topics every PX4 step (period 0), which mirrors the historical lockstep SITL behavior.

12.3 Actuator command extraction

After stepping PX4, the autopilot reads actuator outputs from uORB subscriptions (`UORBOutputs.actuator_motors`; `UORBOutputs.actuator_servos`). It then converts these into fixed-size `ActuatorCommand` arrays and publishes them into the runtime bus (`src/sim/Sources/Autopilot.jl`).

Actuator range semantics. The bridge uses normalized uORB outputs: `actuator_motors` is treated as a duty-like command in $[0, 1]$, and `actuator_servos` as a signed deflection in $[-1, 1]$. These are lockstep ABI conventions (not PWM). If a different uORB topic or scaling is required, the mapping should be changed in the bridge configuration and documented alongside the uORB interface.

12.4 Derived PX4 parameterization: control allocator geometry

When enabled in the aircraft spec (`PX4Spec.derive_ca_params`), the build layer derives `CA_*` control allocator parameters from the multirotor geometry (`src/sim/Aircraft/Build.jl`). For each rotor i with body position $\mathbf{r}_{b,i}$ and propulsor axis $\mathbf{a}_{b,i}$, PX4 expects a *thrust direction* axis; the builder therefore provides $-\mathbf{a}_{b,i}$ (recall section 5.3). The yaw moment coefficient parameter `CA_ROTOR<i>_KM` is set to `km_m` times the rotor reaction torque sign pattern `rotor_dir[i]`.

Caveat. This parameter derivation keeps PX4-side allocation consistent with the Julia-side plant geometry, but it does not configure PX4 mixers or select an airframe; those remain PX4 configuration responsibilities.

12.5 TOML configuration: PX4 bridge, lockstep rates, and uORB interface

The PX4 integration is configured under `[px4]` (`PX4Spec` parsed in `src/sim/Aircraft/TOMLIO.jl`). The same table also contains the lockstep module scheduling rates (`[px4.lockstep]`) and the uORB publish/subscribe interface definition (`[px4.uorb]`).

Example (Iris defaults).

```
[px4]
libpath = "../../../../build/px4_sitl_lockstep/src/lib/px4_lockstep/libpx4_lockstep.so"
mission_path = "../../src/Workflows/assets/missions/simple_mission.waypoints"
derive_ca_params = true
edge_trigger = false

# Optional lockstep-module scheduler configuration (PX4Lockstep.LockstepConfig)
[px4.lockstep]
enable_commander = 0
commander_rate_hz = 100
enable_navigator = 0
navigator_rate_hz = 20
enable_mc_pos_control = 1
mc_pos_control_rate_hz = 100
enable_mc_att_control = 1
mc_att_control_rate_hz = 250
enable_mc_rate_control = 1
mc_rate_control_rate_hz = 250
enable_control_allocator = 1
control_allocator_rate_hz = 250
dataman_use_ram = 1

# uORB bridge configuration can be extended from an asset (e.g. iris_uorb.toml)
[px4.uorb]
pubs = ["vehicle_local_position", "vehicle_attitude", "battery_status"]
subs = ["actuator_outputs"]

# Optional PX4 parameter overrides (applied at init)
[[px4.params]]
name = "SYS_AUTOSTART"
value = 4001

[[px4.params]]
name = "COM_ARM_CHK"
value = 0
```

Parameter mapping.

- `libpath`: shared library path for `libpx4_lockstep` (loaded at runtime).
- `mission_path`: optional PX4 mission/waypoint file path. If absent, PX4 can still run in manual/offboard modes.
- `derive_ca_params`: when `true`, derive control-allocation parameters from the actuation geometry during build (see `Build.derive_control_allocator_params`).
- `edge_trigger`: selects whether mission/RTL requests are treated as edges (one-shot) or levels (held) in the autopilot command logic.

- `[px4.lockstep]`: configures which PX4 modules are stepped in the lockstep loop and at what rates (fields correspond one-to-one with `PX4Lockstep.LockstepConfig`).
- `[px4.uorb]`: declares which uORB topics are published into PX4 (`pubs`) and which are subscribed out of PX4 (`subs`); see Appendix B.
- `[[px4.params]]`: list of {name,value} pairs applied to PX4 parameters at initialization.

13 Aircraft specification and composition

Simulation instances are built from an `AircraftSpec` (`src/sim/Aircraft/Spec.jl`) typically loaded from TOML via `Aircraft.load_spec` (`src/sim/Aircraft/TOMLIO.jl`). The spec layer exists to separate:

- continuous plant integration choices (integrator, contact model),
- environment and timeline definition,
- aircraft component instances (airframe geometry, motors, batteries), and
- runtime boundary configuration (PX4 library path, uORB config, logging/recording).

13.1 TOML specs and inheritance

The repository ships TOML-backed default specs for common workflows (e.g. Iris). Specs support an `extends=[...]` field which deep-merges one or more base specs to form the final configuration (`_load_toml_with_extends`). The merge rules are:

- tables merge recursively,
- arrays replace (they do *not* append),
- scalars replace.

Cycles in the `extends` chain are detected and rejected.

Strict schema checking. When `strict=true` (the default in all shipped workflows), each TOML table is guarded by an explicit allowed-key set (`_known_keys!`). Unknown keys are treated as errors rather than being ignored.

Path resolution. All file paths in the spec (e.g. `px4.mission_path`, `px4.libpath`, battery asset CSVs) are resolved relative to the directory of the root spec file (`_resolve_path`) with `~` expansion. Some fields require the file to exist (e.g. `extends` items), others allow non-existing outputs (e.g. log/record output paths).

Microsecond quantization. Timeline intervals in seconds are converted to integer microseconds with a strict exactness check (section 2.1). If you specify a cadence such as 0.004 seconds, it must be exactly representable as an integer number of microseconds.

TOML block	Julia type	Notes (defaults / build-time behavior)
`schema_version`	`Int`	Must be `1`.
`aircraft`	name + seed	`seed` drives deterministic RNGs (wind uses `seed`, estimator uses `seed+1`).
`home`	`Autopilots.HomeLocation`	Used for NED↔LLA injection into PX4.
`px4`	`PX4Spec`	`libpath` required for `:live`/`:record`; `uorb` contract defines pubs/subs.
`timeline`	`TimelineSpec`	Defines `t_end` and axis cadences; converted to integer microseconds.
`environment`	`EnvironmentSpec`	Live build uses selected wind model; replay build forces `NoWind`.
`scenario`	`ScenarioSpec`	Default builder creates an `EventScenario` with `arm_time` and `mission_time`.
`estimator`	`EstimatorSpec`	`kind=:none` or `:noisy_delayed`; delay defaults to $2\,dt_{ap}$ when unset.
`plant`	`PlantSpec`	Selects integrator and contact model.
`airframe`	`AirframeSpec`	Multirotor geometry + propulsor + rigid-body parameters.
`actuation`	`ActuationSpec`	Maps PX4 channels to physical motors/servos; selects actuator dynamics models.
`power`	`PowerSpec`	Battery models + bus topology and sharing policy.
`sensors`	sensor list	Optional; currently GPS/rangefinder/radar stubs.
`run`	run defaults	Optional defaults for `mode`, recording paths, and CSV logging.

Table 3: High-level TOML schema mapping into `AircraftSpec`. The code treats this schema as part of the simulator contract: unknown keys are rejected in strict mode.

13.2 Assembly into an engine

The build entrypoint constructs an *aircraft instance* containing:

- a fully constructed hybrid timeline (autopilot/wind/log/scenario axes merged into a single event axis),
- an initial plant state x_0,
- the plant RHS functor $f(t, x, u)$ (typically `CoupledMultirotorModel`),
- an integrator instance, and
- a set of runtime sources (scenario, wind, estimator, autopilot) selected by run mode.

The runtime engine then wires these into `Sim.Runtime.Engine`.

13.3 Schema overview (TOML → `AircraftSpec`)

The top-level TOML keys map to strongly-typed sub-structs inside `AircraftSpec` via `spec_from_toml_dict`. table 3 summarizes the main blocks; each block has additional substructure that is validated during parsing.

13.4 Live vs replay builds

The same plant model and engine are used in multiple modes:

- **Live**: sources are stateful components driven by deterministic RNGs (wind, estimator), and PX4 is stepped through the lockstep ABI.
- **Record**: a live run with additional boundary-aligned stream capture.
- **Replay**: live sources are replaced by trace samplers (ZOH/sample-and-hold) so the plant is driven by recorded inputs.

In replay, the environment's wind model is replaced with `NoWind` so that all wind forcing comes from the recorded wind trace and the bus sample-and-hold input.

13.5 Geometry-consistent PX4 configuration

When enabled, the builder derives PX4 control allocator geometry parameters from the multirotor spec (section 12). This prevents a common lockstep failure mode where the controller allocates using a different rotor layout than the plant uses.

Defaults. Many `AircraftSpec` sub-structs have pragmatic defaults. For example, `PlantSpec.integrator=:RK45`. Environment defaults include `EnvironmentSpec.wind=:ou` and `EnvironmentSpec.gravity=:uniform`. The shipped Iris spec asset in section A provides a concrete example.

14 Record → replay and determinism verification

The runtime includes a deliberately small record/replay facility whose sole purpose is to make plant-side changes falsifiable without re-running PX4. A "record" run captures boundary-aligned streams on the engine's integer-microsecond axes. A "replay" run then swaps live sources for deterministic trace samplers (section 2.3), turning the closed-loop PX4 run into an open-loop plant replay driven by the recorded inputs.

14.1 Trace representation

Recorded streams are represented as typed traces indexed by an explicit integer-microsecond axis (`src/sim/Recording/Traces.jl`). The implementation provides three sampling semantics:

- `SampledTrace`: defined only at exact axis times; sampling throws if the query time does not match an axis element.
- `ZOHTrace`: zero-order hold; sampling at time t returns the most recent sample with time $\leq t$.
- `SampleHoldTrace`: identical lookup to `ZOHTrace` but used to communicate "sample-and-hold input" intent (notably wind forcing).

All sampling is a deterministic binary search over the stored axis vector.

14.2 Recorder and axis validation

The default recorder is `Recording.InMemoryRecorder` (`src/sim/Recording/Recorder.jl`). It stores, for each named stream, a vector of timestamps and a vector of values.

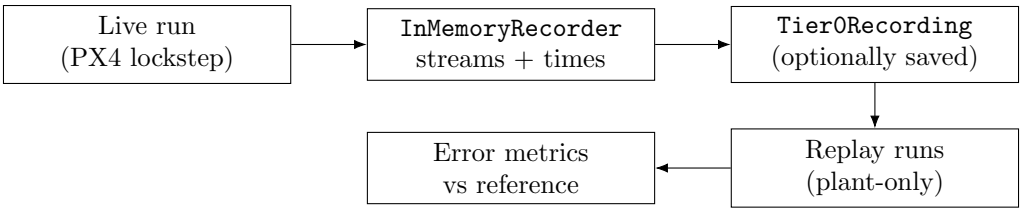

Figure 16: Record/replay workflow: a single PX4 live run records boundary-aligned streams which are then reused to replay the plant under different integrators or plant variants.

Record-time checks. Each stream enforces nondecreasing timestamps (monotone time). It intentionally does *not* require that every recorded timestamp belongs to a particular axis; that contract is validated when a trace is constructed.

Trace-build checks. When converting streams into traces, helpers such as `zoh_trace(rec,name, axis)` validate that the recorded timestamps match the expected engine axis *exactly* (`_require_ axis_match!`). This is a fail-fast guard: "almost aligned" recordings do not silently produce incorrect sample-and-hold behavior.

14.3 What is recorded in `mode=:record`

Record mode is implemented in the canonical engine loop (`src/sim/Runtime/Engine.jl`) via explicit `record!(recorder,name,t_us,value)` calls at boundary stages.

Tier-0 streams. The helper `Recording.tier0_traces` builds a minimal bundle sufficient for plant-only replay:

- `:cmd` (`ActuatorCommand`) recorded on the autopilot axis and replayed with ZOH semantics.
- `:wind_base_ned` (`Vec3`) recorded on the wind axis and replayed as sample-and-hold.
- `:plant`, `:battery`, `:batteries` recorded on the log axis as exact samples.

Scenario streams. The engine can also record scenario outputs (arm/mission/RTL command, landed flag, faults, and optional wind disturbance). Two recording layouts exist (`src/sim/ Recording/Recorder.jl`):

- **Event-axis recording** (keys `*_evt` on `timeline.evt`): enabled by `EngineConfig.record_ faults_evt=true` (default in `Sim.simulate`). This captures scenario changes at the earliest possible hybrid boundary.
- **Scenario-axis recording** (keys without suffix on `timeline.scn`): used when event-axis recording is disabled.

Log snapshots are pre-step. When the log axis is due, the engine records the current plant state and battery telemetry *before* integrating the next interval (see the `due_log` block in `process_ events_at!`). As a result, `:plant[k]` corresponds to the boundary state at `timeline.log.t_us[k]`.

Stream key	Value type	Axis	Trace semantics
`:cmd`	`ActuatorCommand`	`ap`	`ZOHTrace`
`:wind_base_ned`	`Vec3`	`wind`	`SampleHoldTrace`
`:plant`	`PlantState`	`log`	`SampledTrace`
`:battery`	`BatteryStatus`	`log`	`SampledTrace`
`:batteries`	`Vector\{BatteryStatus\}`	`log`	`SampledTrace`
`:ap_cmd_evt`, `:landed_evt`, `:faults_evt`	scenario outputs	`evt`	`ZOHTrace`
(optional) `:wind_dist_evt`	`Vec3`	`evt`	`ZOHTrace`
(optional) `:est`	`EstimatedState`	`ap`	`ZOHTrace`

Table 4: Primary recorded streams and how they are replayed. "Tier-0" refers to the minimal set required to replay plant dynamics without stepping PX4.

14.4 Replay builds

Replay runs are ordinary engine runs where live sources are replaced by trace-based sources:

- `Sources.ReplayAutopilotSource` publishes recorded `bus.cmd` (ZOH on `timeline.ap`).
- `Sources.ReplayWindSource` publishes recorded base wind (sample-and-hold on `timeline.wind`).
- `Sources.ReplayScenarioSource` (optional) publishes recorded `bus.ap_cmd`, `bus.landed`, `bus.faults`, and `bus.wind_dist_ned`.

To avoid double-forcing, the replay environment disables the live wind model (`Environment.NoWind`); all wind forcing comes from the recorded trace plus any replayed disturbance applied at event boundaries (section 2.3).

14.5 Integrator sweep workflow

The workflow `Workflows.RecordReplay.compare_integrators_recording` runs a reference replay and then replays the same recording under a solver sweep:

1. Build replay sources from the recording (Tier-0 traces, plus scenario traces if required).
2. Run a *reference* plant replay on the same `timeline` and `plant0`, recording `:plant` and `:battery` on the log axis.
3. For each candidate solver, rerun the plant replay and compute error series vs the reference trajectory.

The default reference integrator is an RK45 instance with tighter tolerances than the nominal Iris config (`_default_reference_integrator()` in `src/Workflows/RecordReplay.jl`). Error metrics are computed by `Verification.error_series` (`src/sim/Verification.jl`) and reported as max norms over the log-axis samples.

Iris convenience wrapper. `Workflows.RecordReplay.compare_integrators_iris_mission` combines:

- an optional PX4 live record run via `Workflows.simulate_iris_mission(mode=:record)`,
- reconstruction of the replay plant model from the TOML spec, and
- a plant-only replay sweep with optional CSV output.

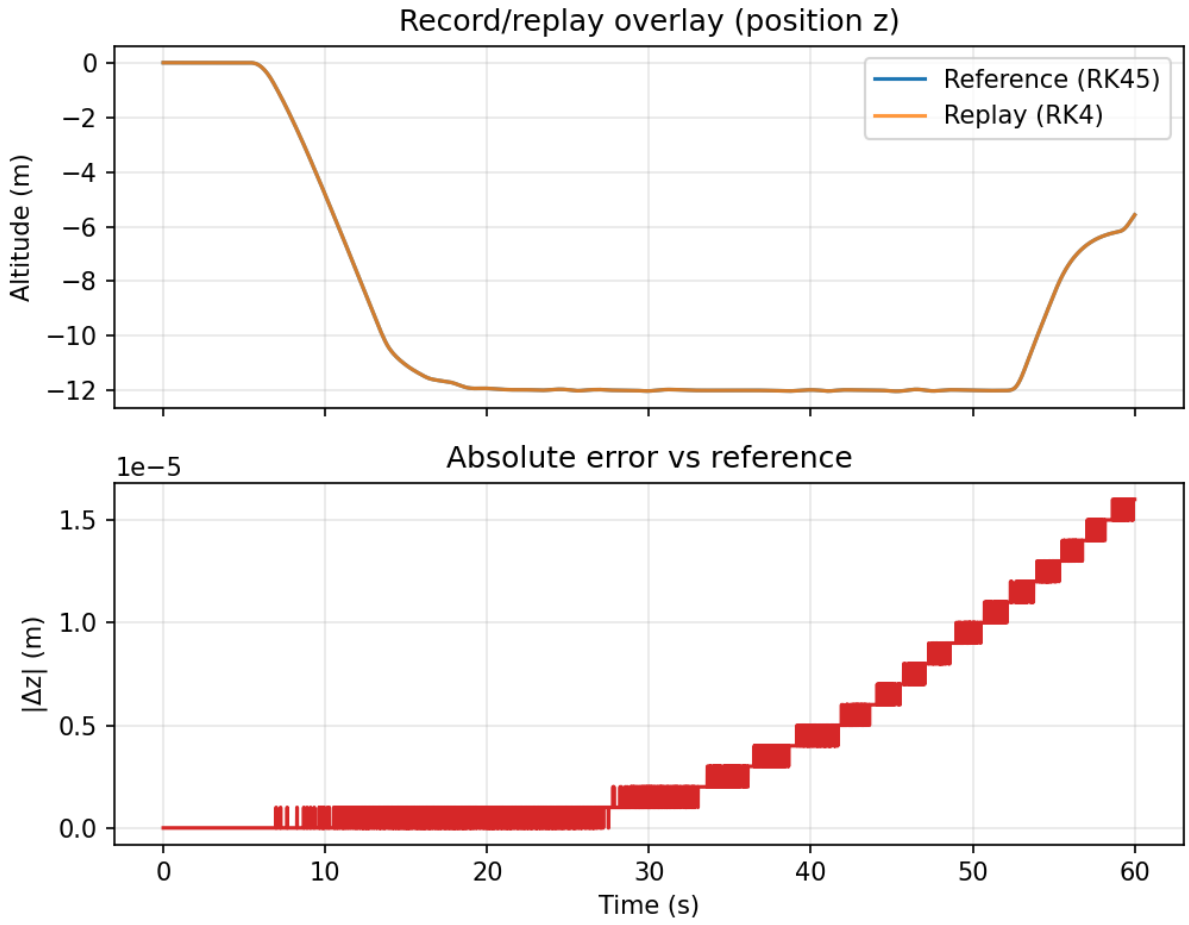

Figure 17: Record/replay overlay for a single axis (position z) comparing the reference replay (RK45) to a replay with RK4.

14.6 Determinism caveats

The recording format is Julia `Serialization` (`Recording.write_recording`/`read_recording`). To reduce foot-guns, a schema version field (`BUS_SCHEMA_VERSION`) is checked at load time. Nevertheless, recordings should be treated as best-effort artifacts: they are intended for within-repository regression and integrator envelope validation, not as a stable long-term interchange format.

Finally, a `Tier0Recording` intentionally does not persist the full plant model or spec. Plant-only replay therefore assumes you rebuild the same `dynfun` (and consistent parameters) used to generate the recording.

14.7 TOML configuration: run modes and recording I/O

Record/replay behavior is selected in the `[run]` block (`RunSpec` in `src/sim/Aircraft/Spec.jl`):

```
[run]
mode = "lockstep" # "lockstep" or "replay"
recording_out = "run.jls" # optional; if set, writes a recording
recording_in = "run.jls" # used only when mode="replay"
```

```
log_csv = "log.csv" # optional; if set, writes a CSV log
```

Semantics.

- `mode`: selects whether PX4 is stepped live (`"lockstep"`) or whether the simulator consumes a previously recorded input/bus stream (`"replay"`).
- `recording_out`: when non-null, the engine serializes the replayable boundary stream at the end of each run.
- `recording_in`: path to a previously recorded stream; required for replay.
- `log_csv`: optional human-readable CSV output independent of the replay stream.

15 Validation and regression checks

The simulator is intended to be reviewable and falsifiable: core claims are backed by executable tests in the repository. This section summarizes the current verification coverage and the contracts it enforces.

15.1 Test harness

The package test entrypoint is `test/runtests.jl`. A typical local invocation is:

```
julia --project -e 'using Pkg; Pkg.test()'
```

The test suite is designed to run without a PX4 shared library. Lockstep runtime smoke tests (`test/integration/lockstep_runtime.jl`) are conditionally executed only when the lockstep library is available (see the `PX4Lockstep.has_lockstep()` gate).

15.2 Analytic integrator verification

The module `Sim.Verification` defines closed-form or invariant-rich reference problems, used in `test/verification/verification_cases.jl`:

- **Simple harmonic oscillator (SHO).** With `dt=0.01` over `T=1.0`, RK4 is required to match the analytic position to $< 10^{-6}$. An additional regression checks that adaptive RK45 with `quantize_us=true` does not drift the requested timebase.
- **Pendulum.** For a small-angle case, RK4 must match the small-angle analytic solution to $< 10^{-5}$ and conserve energy to relative error $< 10^{-4}$ over one period.
- **Circular Kepler orbit.** RK4 must return to the analytic state after one orbit (at $T_{end} = n\,dt$) to $< 10^{-5}$ in position/velocity and conserve energy and angular momentum to $< 10^{-6}$ relative error.
- **Torque-free rigid body.** RK4 must conserve rotational energy to $< 10^{-6}$ relative error, keep quaternion norm within $< 10^{-10}$, and keep inertial angular momentum nearly constant.

These tests make the integrator implementations (section 6) directly falsifiable.

Case	Solver	Max error	Invariant drift
SHO ($\omega_0 = 2\pi$, $T = 1\,\mathrm{s}$, $dt = 0.01$)	RK4	4.27×10^{-8}	$\Delta E/E = 8.54 \times 10^{-8}$
SHO (adaptive)	RK23	9.27×10^{-10}	$\Delta E/E = 1.85 \times 10^{-9}$
SHO (adaptive)	RK45	1.31×10^{-10}	$\Delta E/E = 2.62 \times 10^{-10}$
Pendulum (small angle, $dt = 0.01$)	RK4	3.65×10^{-6}	$\Delta E/E = 2.62 \times 10^{-9}$
Pendulum (adaptive)	RK23	3.65×10^{-6}	$\Delta E/E = 1.19 \times 10^{-8}$
Pendulum (adaptive)	RK45	3.65×10^{-6}	$\Delta E/E = 1.67 \times 10^{-9}$
Kepler (circular, $T = 2\pi$, $dt = 0.01$)	RK4	1.52×10^{-9}	$\Delta E/E = 1.75 \times 10^{-11}$
Kepler (circular, adaptive)	RK23	9.97×10^{-10}	$\Delta E/E = 2.07 \times 10^{-10}$
Kepler (circular, adaptive)	RK45	3.67×10^{-10}	$\Delta E/E = 1.15 \times 10^{-10}$
Rigid body (torque-free, $dt = 0.001$)	RK4	$\lvert\Delta L\rvert = 5.81 \times 10^{-12}$	$\Delta E/E = 4.94 \times 10^{-14}$
Rigid body (torque-free, adaptive)	RK23	$\lvert\Delta L\rvert = 7.55 \times 10^{-10}$	$\Delta E/E = 1.16 \times 10^{-10}$
Rigid body (torque-free, adaptive)	RK45	$\lvert\Delta L\rvert = 5.11 \times 10^{-10}$	$\Delta E/E = 4.76 \times 10^{-11}$

Table 5: Representative analytic/invariant-rich verification results from standalone scripts. "Max error" is the final-state analytic deviation at $T_{end} = n\,dt$ for SHO/pendulum/Kepler, and $|\Delta L|$ is the inertial angular-momentum change for the torque-free rigid-body case.

Solver	pos	vel	att	ω	rotor ω	$\lvert\Delta I\rvert$
RK4	7.52×10^{-5}	4.14×10^{-6}	7.30×10^{-8}	2.24×10^{-7}	1.15×10^{-4}	2.92×10^{-6}
RK23	6.37×10^{-4}	2.96×10^{-5}	2.62×10^{-7}	2.49×10^{-6}	1.61×10^{-3}	6.98×10^{-5}
RK45	0	0	4.21×10^{-8}	0	0	0

Table 6: Representative Iris record/replay integrator sweep: max error vs a reference replay trajectory. Columns correspond to the logged max norms used by `Workflows.RecordReplay.compare_integrators_recording` (position/velocity/attitude/body rates/rotor speed and battery-current error $|\Delta I|$).

15.3 Representative verification results

In addition to the unit-test style checks above, we periodically run standalone verification scripts under `Tools/px4_lockstep_julia/examples/verification/` to obtain concrete error magnitudes for common reference problems. Table 5 summarizes one such run.

The same scripts include reference-compare problems that exercise full `PlantState` integration (rotor speed and battery states) against a tighter RK45 reference, as well as an end-to-end Iris record/replay integrator sweep. One such sweep ($t_{end} = 60\,\mathrm{s}$, $\Delta t_{ap} = 0.004\,\mathrm{s}$, $\Delta t_{wind} = 0.001\,\mathrm{s}$, $\Delta t_{log} = 0.01\,\mathrm{s}$) reported the max errors in table 6 relative to a reference replay.

15.4 Plant-level contract tests

The file `test/verification/verification_contracts.jl` contains deterministic "contract" tests for the full multirotor plant and power system:

- **Free-fall kinematics.** With motors off and no contact, the logged vertical position/velocity must match the closed-form constant-acceleration solution at `atol=1e-7` and maintain quaternion normalization.
- **Hover force-balance (RHS).** A single RHS evaluation is checked by solving for a rotor speed that yields total thrust $\approx mg$ at sea-level density, then asserting vertical acceleration is ≈ 0 at

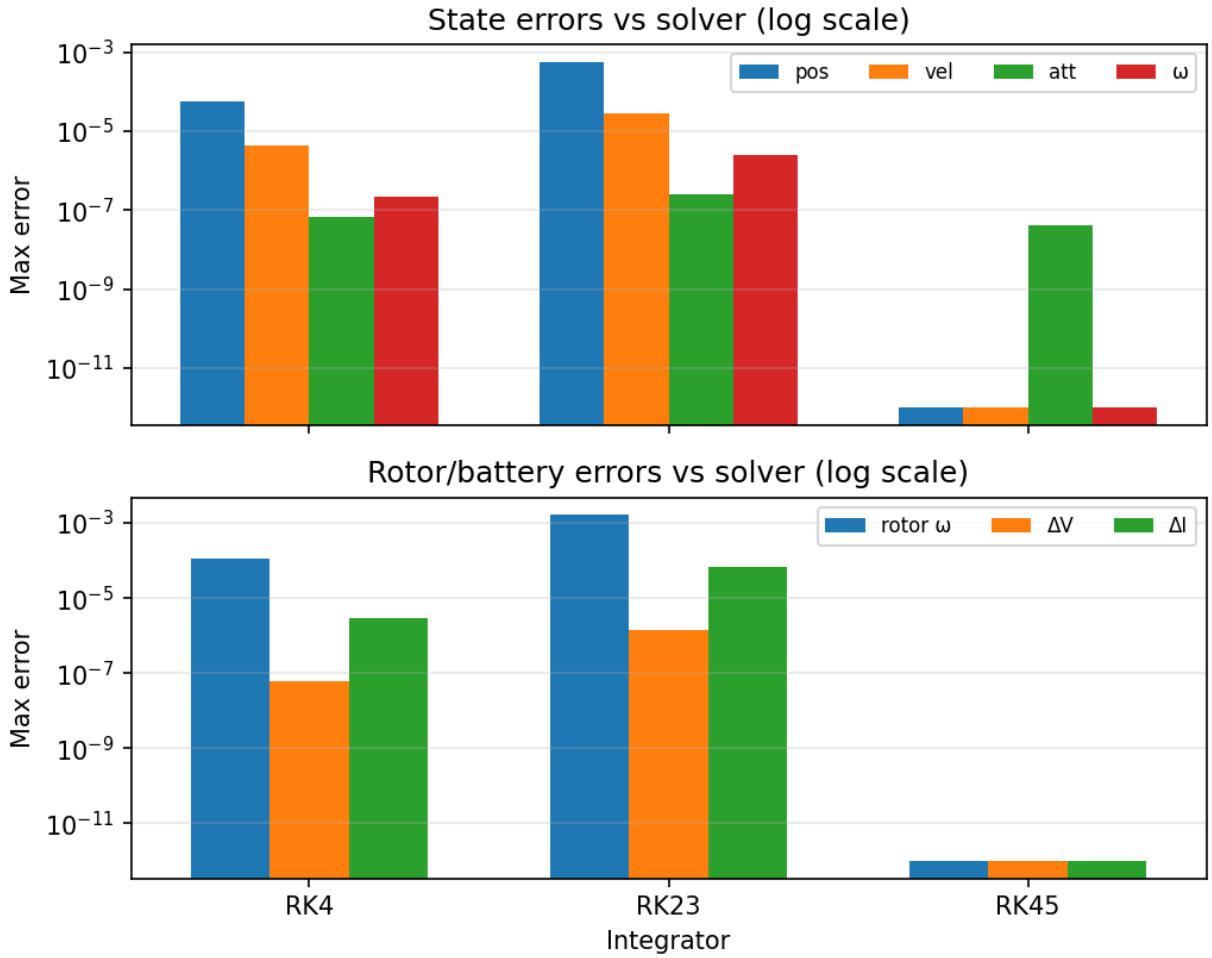

Figure 18: Integrator sweep errors (log scale) for the same Iris record/replay run summarized in table 6.

identity attitude.

- **Power-bus solve fixed point.** A deterministic sweep over randomized rotor speeds/duties checks the bus-voltage solver returns a consistent fixed-point residual.
- **State sanity.** Logged states are asserted finite and within physical bounds (e.g. SOC $\in [0, 1]$, rotor speeds non-negative up to tolerance).

15.5 Record/replay invariants

The record/replay machinery is covered by `test/verification/record_replay_engine.jl`. These tests exercise:

- axis-matching: building traces from a recording throws if recorded timestamps do not match the expected engine axis exactly,
- serialization schema enforcement through `BUS_SCHEMA_VERSION`, and
- replay determinism for Tier-0 streams (commands and wind) under multiple integrators.

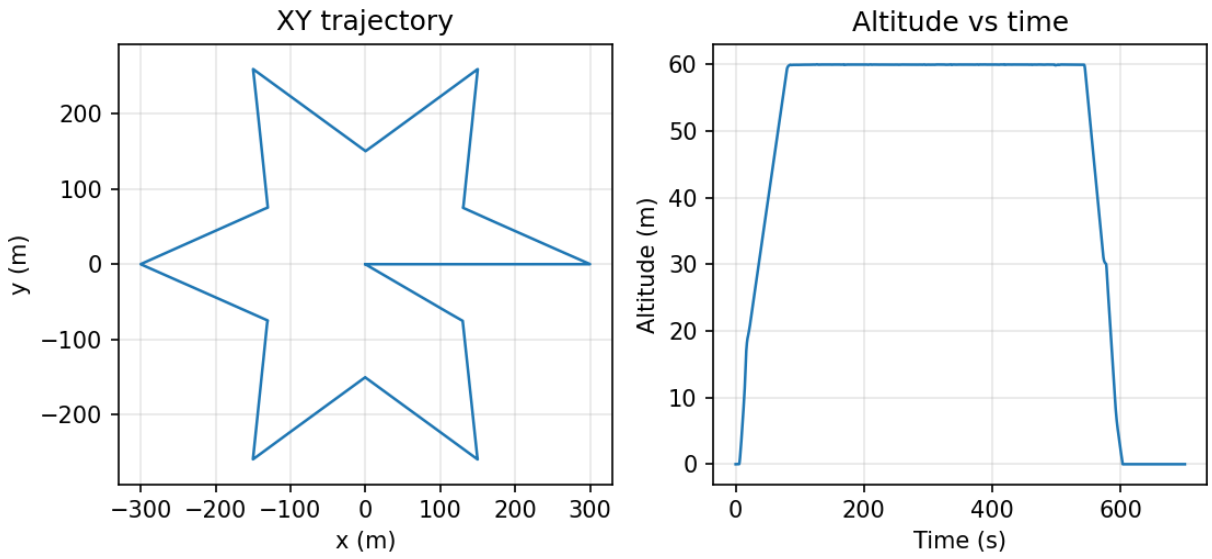

Figure 19: Representative closed-loop Iris mission trace from a lockstep run (XY trajectory and altitude vs time). This is provided as a qualitative sanity check rather than a strict regression.

15.6 uORB injection tests

The uORB injection scheduler and default state injector are covered by `test/uorb/uorb_injection.jl`, including multi-instance battery publishing behavior and the periodic "due" logic.

15.7 Gaps and recommended extensions

The current suite does not yet provide strong, closed-loop PX4 trajectory validation inside this repository (the lockstep runtime tests are intentionally minimal and are skipped when the PX4 shared library is unavailable). In practice, the most useful regression to add is a canonical PX4 mission recorded once and replayed under a solver sweep (section 14.5) with a saved CSV error summary.

16 Extension points

Although the default Iris workflow is "batteries + multirotor plant + wind + PX4 lockstep", the codebase is structured around a small number of explicit interfaces. Extending the simulator is primarily a matter of implementing additional methods for these interfaces rather than modifying the engine.

16.1 Discrete-time sources

All discrete-time components that publish into the canonical signal bus (scenario, wind, estimator, autopilot) implement the runtime protocol:

```
Runtime.update!(src,bus,plant_state,t_us)
```

The default fallback is a loud error (`Runtime.Engine.jl`) to avoid silent mis-wiring.

Sources that introduce additional event boundaries (typically scenarios) may also extend:

```
Runtime.event_times_us(src,t0_us,t_end_us)
```

These event times are merged into the hybrid timeline by `Runtime.build_timeline_for_run`.

Determinism guideline. Sources are the intended home for randomness. They should draw randomness exclusively from explicit RNG state (seeded from `AircraftSpec.aircraft.seed`) and should never consult wall-clock time.

16.2 Plant models and outputs

A plant model is a callable RHS functor used by the integrator:

```
dx=model(t::Float64,x::PlantState,u::PlantInput)
```

The engine treats the RHS as pure (no side effects) and therefore assumes it is deterministic given (t, x, u).

Plants may optionally implement additional hooks (`src/sim/PlantInterface.jl`):

- `plant_outputs(model,t,x,u)`: compute algebraic telemetry (forces, thrust, power, contact info) for logging and bus publication at event boundaries.
- `plant_project(model,x)`: enforce invariants after integration (e.g. quaternion renormalization, rotor speed clamping, contact post-stabilization).
- `plant_on_autopilot_tick(model,x,cmd)`: boundary-time discontinuities that must occur exactly at autopilot ticks (e.g. snapping `DirectActuators` outputs to the newly published command).
- `plant_integrate_interval(model,integrator,t0_us,x0,u,dt_us)`: optional hybrid interval stepping that can localize internal events (e.g. contact time-of-impact) and apply an impact map inside the interval.

16.3 Contact models

Ground/contact interaction is factored as a contact model (`src/sim/Contacts.jl`) attached to `PlantSpec.contact`. New models subtype `Contacts.AbstractContactModel` and (at minimum) implement a contact-to-world-force mapping at the vehicle COM:

```
F_contact_ned=Contacts.contact_force_ned(model,rb_state,t)
```

where `rb_state` is the current rigid-body state (NED position/velocity + attitude + body rates). Models may additionally implement

```
ci=Contacts.contact_info(model,rb_state,t)
```

to expose a `ContactInfo` snapshot (gap/penetration, contact mode, normal/friction force estimates) for logging and diagnostics.

For the unilateral constraint model `FlatGroundConstraintContact`, the normal reaction depends on the *unconstrained* acceleration; it is therefore evaluated with the overload

```
F_contact_ned=Contacts.contact_force_ned(model,rb_state,t,mass,a_free_ned)
```

used by the plant outputs layer.

Hybrid touchdown/bounce handling is implemented at the plant level via `plant_integrate_interval(...)` (e.g. time-of-impact localization + impact map for flat-ground contact). The default Iris spec uses `flat_ground_constraint` contact.

16.4 PX4 uORB injection sources

PX4 lockstep integration is mediated through a uORB injector. See `src/sim/Autopilots/UORBInjection.jl`. The injector owns a list of `AbstractUORBInjectionSource` objects. Each source can publish at a declared integer-microsecond cadence via:

```
inject_due!(src,handle,ctx,t0_us)
```

A convenience implementation `PeriodicUORBInjection` is provided: users supply a topic-specific message builder `builder(ctx)` and configure its `period_us` and `phase_us`. The default Iris workflow uses the built-in state injector `build_state_injection_injector`.

17 Related work and positioning

This paper is primarily an implementation description, but it is still useful to position the design against common autopilot-in-the-loop practice. We keep this section intentionally high-level in the current draft; a camera-ready version should add formal citations.

17.1 SITL and lockstep autopilot integration

PX4 SITL deployments commonly pair the autopilot with an external simulator that advances "time" in some form and exchanges state/actuator topics. In many stacks the simulator attempts to run in (scaled) real time, and the autopilot is advanced in a loop that is ultimately coupled to wall-clock scheduling. `PX4Lockstep.jl` instead treats PX4 as a discrete-time component stepped on an injected clock: the simulation engine never advances PX4 by wall time, and the boundary ordering is part of the explicit semantics (section 2.3).

17.2 Physics engines vs. reviewable determinism

High-fidelity physics engines and robotics simulators provide rich contact and aerodynamics models, but they frequently trade away strict repeatability due to internal multi-threading, nondeterministic collision ordering, and floating-point reductions. The design goal here is deliberately different: to make the execution model and the determinism boundary reviewable, and to keep the "truth" models small enough that equations, defaults, and corner cases can be documented code-faithfully.

17.3 Deterministic hybrid simulation and record/replay

The union-event-axis execution model is closely related in spirit to deterministic discrete-event simulation: subsystems publish changes only at explicitly scheduled boundaries and the continuous state advances deterministically between them. Record/replay then becomes a natural scientific tool: a closed-loop run can be reduced to boundary-aligned input traces and replayed as an open-loop plant experiment (section 14). This mirrors "record and replay" ideas used in systems debugging, but the emphasis here is on plant-side falsifiability: any divergence under replay is attributable to plant/integrator/model changes, not controller behavior.

Citations. Formal citations are intentionally deferred in this draft. A camera-ready revision should cite: (i) canonical PX4 SITL simulators and lockstep interfaces, (ii) deterministic hybrid simulation frameworks/discrete-event semantics, and (iii) record/replay systems for reproducibility and debugging. These citations are not load-bearing for understanding the implementation described here.

18 Nonstandard modeling choices and rationale

This simulator is optimized for deterministic regression and rapid iteration. A few modeling choices intentionally diverge from common aerospace references; they are not "wrong," but they should be read as pragmatic tradeoffs rather than canonical physics. The list below records these choices and the intended future direction.

18.1 Local NED-to-LLA approximation

The NED-to-LLA conversion used for PX4 topic injection is a flat-Earth approximation with a fixed $\cos(\varphi_0)$ longitude scale and linear altitude (section 5.2). This is adequate for local missions (order $\sim$10 km from home) and deterministic regression, but it is not suitable for large-area navigation or high-latitude operations. A future extension could substitute a proper geodesic conversion or an Earth-centered frame.

18.2 Turbulence model: OU vs. Dryden/Von Kármán

The wind model uses an Ornstein–Uhlenbeck (OU/AR(1)) process (section 10.3) rather than a Dryden or Von Kármán spectral model. OU is chosen because it is simple, strictly Markov, and deterministic under fixed random seeds, making it attractive for regression. It does not attempt to match turbulence power spectra used in certification-oriented literature. If spectral fidelity becomes a goal, Dryden/Von Kármán parameterizations should be added as alternative wind sources.

18.3 State injection instead of raw sensor simulation

The default lockstep bridge injects filtered state estimates into PX4 rather than simulating raw sensors and running an onboard estimator (section 12, section 19). This provides stable, deterministic closed-loop regression but does not exercise estimator failure modes or sensor bandwidth limitations.

18.4 Simplified contact and aerodynamics

Ground contact is modeled as a single COM contact with a hybrid impact map and an impulse-based grounded solve (section 10.4). This is in the family of time-stepping / complementarity-inspired rigid-contact methods (e.g., Stewart–Trinkle 1996; Anitescu–Potra 1997), but implemented in a minimal single-contact form for determinism. Aerodynamics are linear drag only (section 7). These choices prioritize stability and determinism; they are not intended to represent landing gear dynamics or aerodynamic performance.

18.5 Ad-hoc prop inflow attenuation

The propeller inflow correction uses a simple μ^2 attenuation $f(\mu) = 1/(1 + k\mu^2)$ (section 8.2). This is a pragmatic shaping term (not a blade-element or momentum-theory model) and ignores the sign of axial velocity. It therefore cannot represent windmilling, vortex-ring, or braking regimes.

19 Known limitations and improvement targets

This section lists limitations that are *explicit in the code* and therefore matter when interpreting results.

19.1 State injection rather than raw sensor simulation

The default PX4 integration uses "state injection" uORB topics published at every PX4 step (`src/sim/Autopilots/UORBInjection.jl`), not raw IMU/mag/baro/GPS measurements. The estimator layer therefore models "EKF output error" rather than sensor physics.

19.2 Aerodynamics are intentionally minimal

The rigid-body drag model is linear and isotropic (`Vehicles.GenericMultirotor`); it does not include lift, induced drag, rotor/airframe aerodynamic coupling, blade flapping, or ground effect. Propulsor inflow is captured only through a simple μ^2 attenuation factor on thrust/torque (section 8.2).

19.3 Contacts are COM forces only

The shipped ground models (`FlatGroundContact` and `FlatGroundConstraintContact`) apply contact at the vehicle center of mass (COM) and therefore produce no moments. While `FlatGroundConstraintContact` includes deterministic hybrid impact handling (time-of-impact localization + restitution/impact friction) and a fixed-step grounded impulse solve for stable resting contact, it is still not a landing-gear or multi-point contact model. Energy and angular-momentum conservation are not enforced (normal velocity is reset, tangential velocity is clamped by Coulomb bounds, and there is no angular impulse). Terrain heightfields and contact-induced torques remain out of scope for the current contact layer.

19.4 No regeneration / no charging

The BLDC current model clamps negative current to zero, and the battery model integrates only discharge current ($I \leftarrow \max(0, I)$). There is no regenerative braking, motor back-driving into the bus, or battery charging model.

19.5 Power-network topology is intentionally simple

`PowerNetwork` supports only "battery belongs to a bus" wiring with per-bus constant-power avionics loads. Cross-feed, diode OR-ing, and richer circuit effects are intentionally out of scope. Additionally, avionics constant-power load is approximated as a constant current during the bus-voltage solve (section 9.5).

19.6 Static scenario event discovery

The hybrid timeline can include scenario `AtTime` events discovered from a static scheduler. Dynamically inserting new events at runtime is not supported by the current timeline builder.

19.7 Geodesy and gravity approximations

The NED↔LLA conversion used for PX4 topic injection is a local flat-Earth approximation (fixed $\cos(\varphi_0)$ longitude scale and linear altitude). It is intended for missions within $\sim$10 km of the home origin. `SphericalGravity` treats local down as radial with $r \approx r_0 - z$. These are appropriate for short-range missions around the home origin but not for large-area navigation studies.

19.8 Floating-point and environment-dependent determinism

While the runtime semantics are deterministic at the event-scheduling layer (integer timebase, fixed boundary ordering), bitwise-identical results are not guaranteed across different floating-point environments. Multi-threaded dependencies, different `libm` implementations, CPU instruction sets (e.g. fused multiply-add behavior), or dependency version drift can all change rounding and therefore plant trajectories. The project currently targets repeatability *within a controlled environment*; the threat model and recommended controls are stated explicitly in section 3.4.

A Default Iris spec (TOML assets)

The repository ships a TOML-first default Iris configuration:

- `src/Workflows/assets/aircraft/iris_default.toml`: mission timing, plant/environment, airframe, actuation, and power.
- `src/Workflows/assets/aircraft/iris_uorb.toml`: the uORB interface contract (topics to publish/subscribe).

The default spec uses `extends=["iris_uorb.toml"]`; the loader deep-merges these files (section 13).

This appendix summarizes the key resolved parameters of the default Iris spec.

A.1 Home origin and PX4 configuration

Field	Value
`home.lat_deg`	47.397742
`home.lon_deg`	8.545594
`home.alt_msl_m`	488.0 m
`px4.mission_path` (relative to `iris_default.toml`)	`../missions/simple_mission.waypoints`
`px4.edge_trigger`	`false`
`px4.derive_ca_params`	`true`
`px4.libpath`	*unset in the shipped asset (required for live/record)*

Table 7: Home origin and PX4 configuration defaults. Relative paths resolve against the spec file directory.

A.2 Timeline, plant, and environment

A.3 Scenario and estimator

A.4 Rigid body, geometry, and propulsion

A.5 Actuation mapping and power network

uORB contract. The default Iris uORB publish/subscribe set is defined in `iris_uorb.toml`. The resulting state-injection topics and their units/frames are summarized in section B.

PX4 lockstep field	Value
dataman_use_ram	1
enable_commander	0
commander_rate_hz	100
navigator_rate_hz	20
mc_pos_control_rate_hz	100
mc_att_control_rate_hz	250
mc_rate_control_rate_hz	250
enable_control_allocator	1
control_allocator_rate_hz	250

Table 8: Lockstep module rates from [px4.lockstep].

Field	Value
timeline.t_end_s	20.0 s
timeline.dt_autopilot_s	0.004 s
timeline.dt_wind_s	0.001 s
timeline.dt_log_s	0.01 s
plant.integrator	RK45
plant.contact	flat_ground_constraint
environment.atmosphere	isa1976
environment.gravity	uniform, $g = 9.806\,65\,\mathrm{m/s^2}$
environment.wind	ou
environment.wind_mean_ned	$(0, 0, 0)\,\mathrm{m/s}$
environment.wind_sigma_ned	$(1.5, 1.5, 0.5)\,\mathrm{m/s}$
environment.wind_tau_s	3.0 s

Table 9: Default timeline, plant selection, and environment parameters.

Field	Value
scenario.arm_time_s	1.0 s
scenario.mission_time_s	2.0 s
estimator.kind	noisy_delayed
estimator.pos_sigma_m	$(0.02, 0.02, 0.02)\,\mathrm{m}$
estimator.vel_sigma_mps	$(0.05, 0.05, 0.05)\,\mathrm{m/s}$
estimator.yaw_sigma_rad	0.01
estimator.rate_sigma_rad_s	$(0.005, 0.005, 0.005)\,\mathrm{rad/s}$
estimator.bias_tau_s	50.0 s
estimator.rate_bias_sigma_rad_s	$(0.001, 0.001, 0.001)\,\mathrm{rad/s}$
estimator.delay_s	0.008 s
estimator.dt_est_s	0.004 s

Table 10: Default scenario and estimator parameters.

Field	Value
`airframe.mass_kg`	$1.5\,\mathrm{kg}$
`airframe.inertia_diag_kgm2`	$(0.029125, 0.029125, 0.055225)\,\mathrm{kg\,m^2}$
`airframe.linear_drag`	$0.05\,\mathrm{N/(m/s)}$
`airframe.angular_damping`	$(0.02, 0.02, 0.01)$
Rotor positions $\mathbf{r}_{b,i}$ (m)	$\{(0.1515, 0.2450, 0),\ (-0.1515, -0.1875, 0),$ $(0.1515, -0.2450, 0),\ (-0.1515, 0.1875, 0)\}$
Rotor axes $\mathbf{a}_{b,i}$	$(0, 0, 1)$ for all four
`propulsion.km_m`	0.05
`propulsion.V_nom`	$12.0\,\mathrm{V}$
`propulsion.rho_nom`	$1.225\,\mathrm{kg/m^3}$
`propulsion.rotor_radius_m`	$0.127\,\mathrm{m}$
`propulsion.inflow_kT`, `inflow_kQ`	8.0, 8.0
`propulsion.thrust_calibration_mult`	2.0
`propulsion.rotor_dir`	$(+1, +1, -1, -1)$
`esc.eta`	0.98
`esc.deadzone`	0.02
`motor.kv_rpm_per_volt`	920
`motor.r_ohm`	$0.25\,\Omega$
`motor.j_kgm2`	$1.2 \times 10^{-5}\,\mathrm{kg\,m^2}$
`motor.i0_a`	$0.6\,\mathrm{A}$
`motor.viscous_friction_nm_per_rad_s`	2.0×10^{-6}
`motor.max_current_a`	$60\,\mathrm{A}$

Table 11: Default rigid-body, geometry, and propulsion parameters.

Field	Value
Motor channel map	{`motor1` $\mapsto$ 1, `motor2` $\mapsto$ 2, `motor3` $\mapsto$ 3, `motor4` $\mapsto$ 4}
`motor_actuators.kind`	`direct`
`servo_actuators.kind`	`direct`
`power.share_mode`	`inv_r0`
`battery.model`	`thevenin`
`battery.capacity_ah`	$5.0\,\mathrm{Ah}$
`battery.soc0`	1.0
`battery.ocv`	linear: $10.8 \to 12.6\,\mathrm{V}$ over SOC $0 \to 1$
`battery.r0`	$0.020\,\Omega$
`battery.r1`,`battery.c1`	$0.010\,\Omega$, $2000\,\mathrm{F}$
`battery.min_voltage_v`	$9.9\,\mathrm{V}$
`battery.low/crit/emerg`	0.15 / 0.10 / 0.05
Bus wiring	{`main` : `bat1` $\to$ `motor1..4`}, avionics load $0\,\mathrm{W}$

Table 12: Default actuation mapping and power-network parameters (single bus, single battery).

B PX4 uORB injection contract

This appendix documents the concrete uORB messages injected by the simulator into PX4 in lockstep mode. The implementation lives in:

- `src/sim/Autopilots/UORBBridge.jl` (message packing and uORB bridge), and
- `src/sim/Autopilots/UORBInjection.jl` (publish scheduling and state-to-message mapping).

B.1 Injection call site

PX4 is stepped once per autopilot tick through `Autopilots.autopilot_step`. For each tick, the simulator:

1. builds a `PX4StepContext` snapshot containing time, rigid-body state, estimated yaw, geodetic position, autopilot command, and battery telemetry,
2. calls `inject_due!` to publish any uORB topics that are scheduled for the current time, and
3. calls `step_uorb!` to advance PX4's internal uORB graph by one lockstep step.

B.2 Default Iris interface

The default Iris assets define the uORB interface contract in `iris_uorb.toml`. The *publisher* list describes what the simulator injects into PX4; the *subscriber* list describes what the simulator reads back from PX4 after stepping.

Injected uORB topics (default Iris). The default state injector publishes each configured topic *every PX4 step* (period = 0). table 13 lists the injected topics and their primary semantics.

Subscribed uORB topics (default Iris). After each PX4 step, the bridge reads a fixed set of uORB subscriptions to construct the autopilot output bundle `UORBOutputs`. The default Iris list is:

> `torque_sp`, `thrust_sp`, `actuator_motors`, `actuator_servos`, `attitude_sp`, `rates_sp`, `mission_result`, `vehicle_status`, `battery_status`, `trajectory_setpoint`.

Commander-lite topics. When Commander is disabled, the lockstep C runtime publishes `vehicle_status`, `vehicle_control_mode`, and `actuator_armed` based on the high-level command inputs and `mission_result`. These topics are not part of the Julia injection set.

Actuator output normalization. The simulator interprets `actuator_motors` as normalized ESC duty commands in $[0, 1]$ and `actuator_servos` as normalized control-surface deflections in $[-1, 1]$. These are the lockstep ABI conventions used by `ActuatorCommand`; they are not PWM values. If a different uORB topic or scaling is desired (e.g. reversible motors), the mapping should be adjusted in the bridge and documented in the interface config.

B.3 State-to-message mapping details

This section highlights the non-obvious mapping choices that matter for correctness.

Publisher key	uORB msg type	Notes (units / frames)
`battery_status`	`BatteryStatusMsg`	Multi-instance support; uses `ctx.batteries`.
`vehicle_attitude`	`VehicleAttitudeMsg`	Quaternion q_{bn} (Body→NED), ω_{body} in rad/s.
`vehicle_local_position`	`VehicleLocalPositionMsg`	p_{ned} (m), v_{ned} (m/s), yaw (rad).
`vehicle_global_position`	`VehicleGlobalPositionMsg`	Lat/Lon (deg), alt_msl (m).
`vehicle_angular_velocity`	`VehicleAngularVelocityMsg`	Body rates (rad/s).
`vehicle_land_detected`	`VehicleLandDetectedMsg`	Uses `ctx.landed`.
`home_position`	`HomePositionMsg`	Publishes home LLA; gated on finite home values.
`geofence_status`	`GeofenceStatusMsg`	Sets `geofence_violated=false`.

Table 13: Default Iris uORB publishers injected by the simulator (`iris_uorb.toml`). Unless overridden by custom injection sources, all default topics publish every PX4 step.

Attitude quaternion convention. `VehicleAttitudeMsg.q` is populated with q_{bn} (body→NED), ordered as (w, x, y, z) and normalized at parse-time in the TOML loader. See section 5 for the simulator frame conventions.

Local position reference. `VehicleLocalPositionMsg.ref_lat/ref_lon/ref_alt` are chosen based on whether commander-in-loop is enabled. When `enable_commander==0` (the Iris default), the reference is forced to the configured home location so PX4 receives a consistent local-frame origin.

Battery injection and instances. Battery status injection supports multiple uORB instances. If the interface config contains multiple `battery_status` publishers with instances $0, 1, \ldots$, the injector publishes one message per instance. Instance i maps to `ctx.batteries[i+1]`; missing batteries are rejected.

B.4 Publish scheduling and cadence constraints

Each injected topic is driven by an `AbstractUORBInjectionSource` with parameters:

- `period_us`: publish period in microseconds (`0` means "every PX4 step"),
- `phase_us`: phase offset relative to the injector's `t0_us` (captured on first use).

For non-zero periods, a publish is *due* at time t when $(t - t_0 - \phi) \bmod P = 0$ (see `_is_due` in `UORBInjection.jl`).

Cadence validation. When running a live PX4 autopilot, `Sim.simulate` checks that periodic injection sources are compatible with the autopilot cadence:

- the autopilot step Δt_{ap} must not be slower than the minimum declared injection period, and
- each declared injection period must be an integer multiple of Δt_{ap}.

If an injection period is not a multiple of the autopilot cadence and strict checking is disabled, the simulator warns that scheduled publishes will be skipped (effective period becomes the least common multiple of P and Δt_{ap}).

Default Iris behavior. The built-in state injector sets all default topic periods to `0`, so the injection schedule does not constrain Δt_{ap} for the default Iris workflow.

C Run context for reported figures

This appendix records the configuration used for the replay-only runtime figure (figure 5). The goal is to make the reported numbers reproducible without overloading the figure caption.

C.1 Replay-only realtime factor (700 s mission)

Field	Value
Spec path	`examples/specs/iris_example_3s2p_10ah_surface_rc_thermal_lockstep.toml`
Run mode	Replay-only sweep (no PX4 calls during replay)
Recording source	`examples/replay/out_long/iris_long_tier0.jls`
t_{end}	700 s
Δt_{ap}	0.004 s
Δt_{wind}	0.001 s
Δt_{log}	0.01 s
Integrators swept	RK4, RK23, RK45 (nominal tolerances)
Reference integrator	RK45 (tight tolerances; `defaultreferenceintegrator`)
Contact model	`flat_ground_constraint` (hybrid)
Battery model	Thevenin + surface R0(s,T), thermal enabled

Table 14: Configuration context for the replay-only realtime factor plot.

The RTF values in figure 5 are computed from the recorded mission duration (t_{end}) and the measured wall time of each replay. Because the replay sweep does not call into PX4, these timings reflect only the Julia-side plant and engine cost.